# CS C280 Homework 2: Facial Keypoint Detection

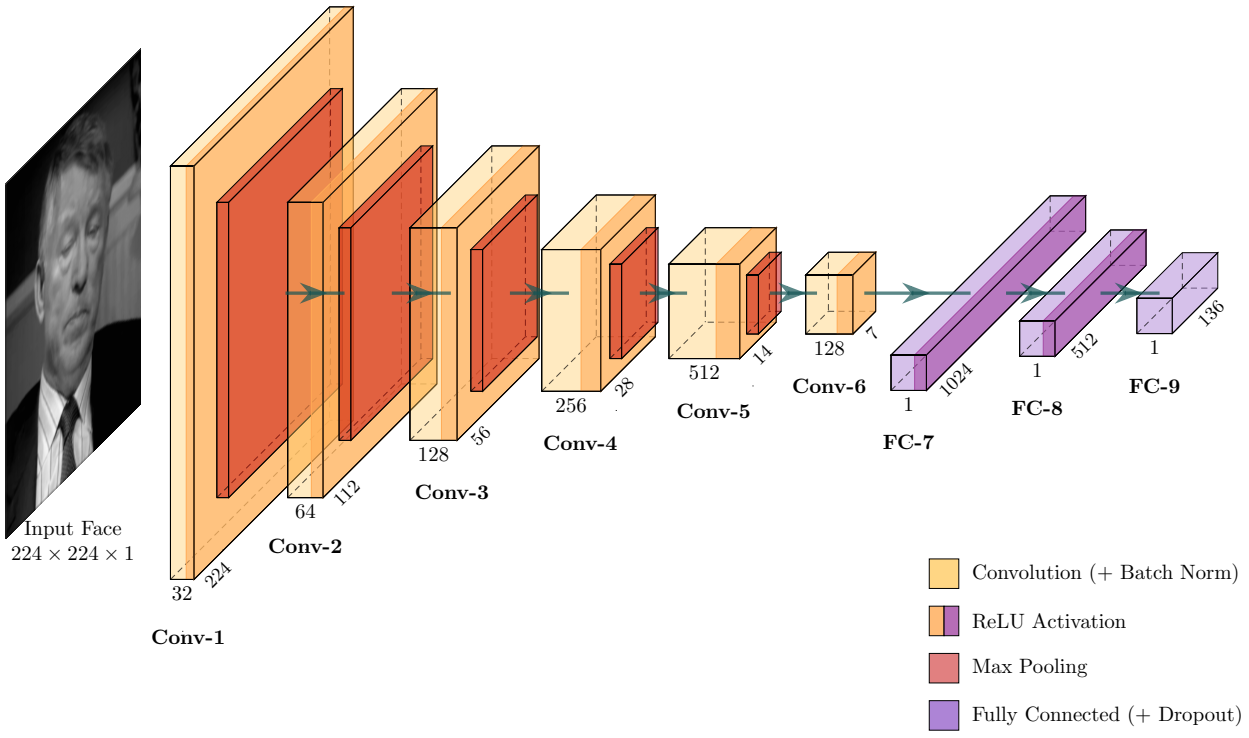
Joel E. Castro Hernandez

February 24, 2025

## 1 Introduction

Facial keypoint detection is a fundamental computer vision task that involves identifying the locations of important facial landmarks such as eyes, nose, and mouth corners in images. This task has numerous applications including face recognition, emotion analysis, augmented reality, animation, medical diagnosis, and more. In this assignment, we implement and compare different approaches for facial keypoint detection, ranging from direct coordinate regression to heatmap-based detection methods. We will also explore how transfer learning from pretrained models can improve performance on this task. The assignment specs and dataset can be found here: https://drive.google.com/drive/u/1/folders/1ODHO8sP78qTawGy9MkReZyBjZvA8Udc7.

## 2 Part 1: Direct Coordinate Regression




FacialKeypointCNN Architecture

### 2.1 Model Description and Implementation

The FacialKeypointCNN's inputs consist of $224 \times 224 \times 1$ images, and the output predictions are $1 \times 136$ tensors returned as $68 \times 2$ tensors (68 keypoints represented as $(x, y)$ coordinates). The architecture consists

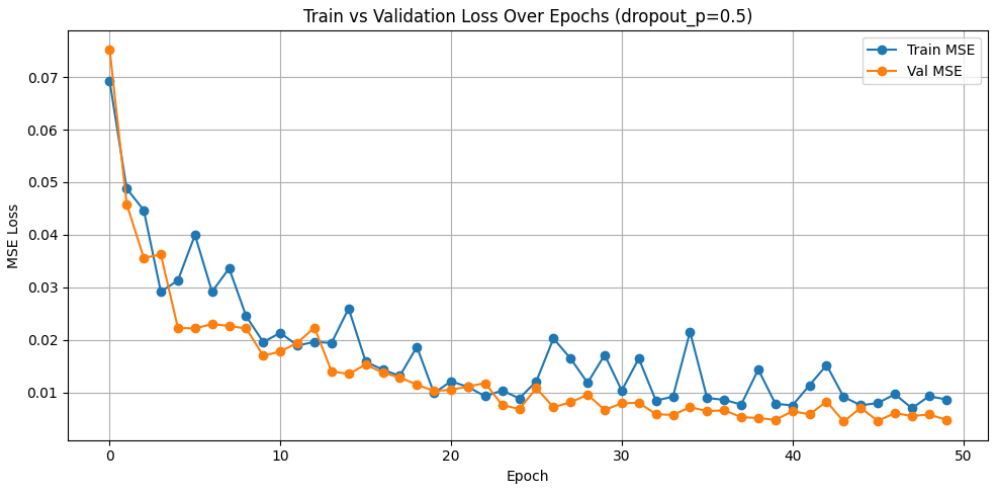
of 9 layers (6 convolutional and 3 fully connected) as detailed in the figure above 2 (inspired by the CIFAR10 Convolutional Neural Network used in the official PyTorch documentation [3]). To maintain the same input to output dimensions during convolution, we maintained a kernel size of 3, a padding of 1, and a stride of 1 for all convolutional layers. The spatial dimensions of the output volume were calculated using the standard formula:

$$\text{output\_dim} = \frac{\text{input\_dim} - \text{kernel\_size} + 2 \times \text{padding}}{\text{stride}} + 1 \tag{1}$$

By preserving the input dimensions during the convolution step, this configuration allowed me to exactly halve the spatial dimensions strictly through the $2 \times 2$ max pooling operations. Moreover, the network made use of the dropout technique to prevent overfitting [4], utilizing a $p = 0.5$ (50% chance of neuron being zeroed out) by default.

## 2.2 Training and Validation

### 2.2.1 Hyperparameters




Training and Validation Loss curves for Direct Coordinate Regression ($p = 0.5$).

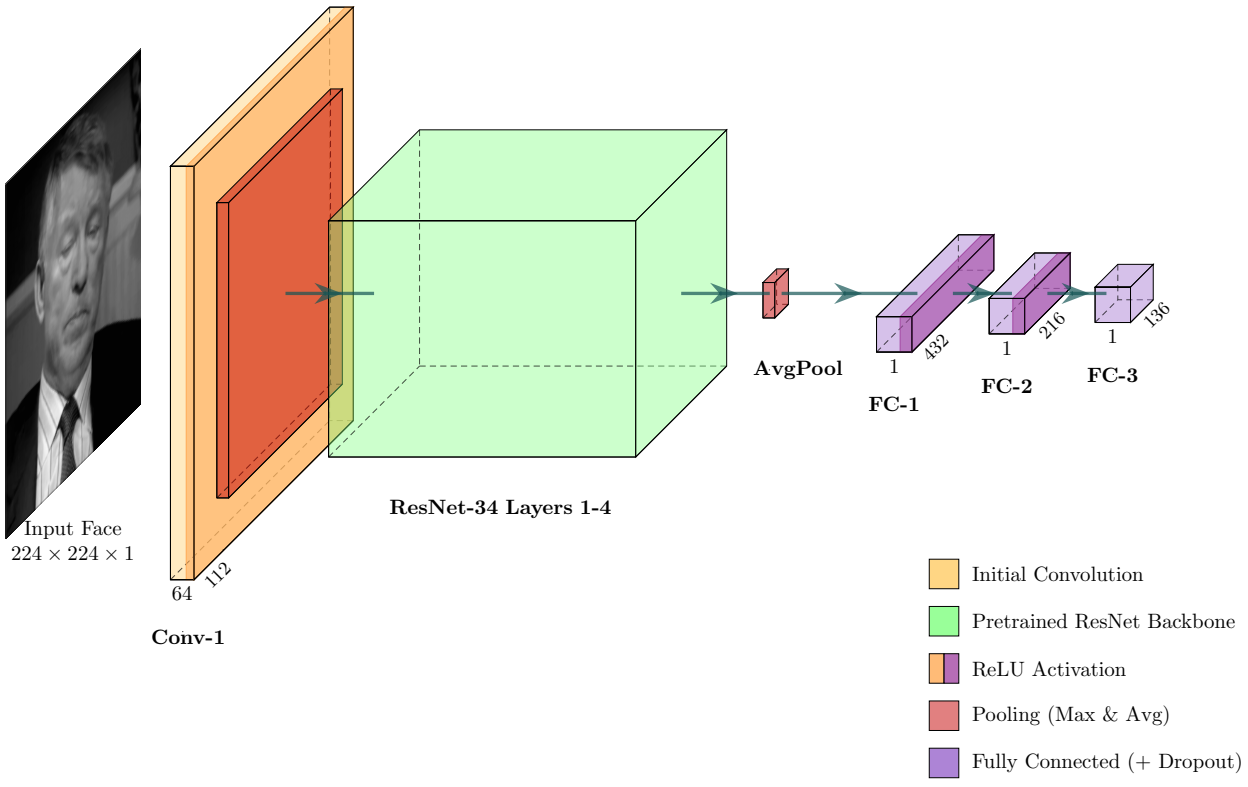
During training, we use Smooth L1 Loss, a learning rate of $1 \times 10^{-3}$, and hyperparameter tune through 50 epochs, and three values of $p \in \{0.0, 0.3, 0.5\}$. As the number of epochs went on, both the training and validation loss trended downward; this tells us that the model likely would benefit from further epochs. Moreover, the $p$ tuning found that $p = 0.0, 0.3$ (6.4) both see the loss drop quicker, but the validation loss is more prone to being greater than the training loss (i.e. a marker of overfitting). Therefore, for longer training loops, maintaining a higher $p$, such as $p = 0.5$, seems promising for preventing overfitting (as expected from [4]).

## 2.3 Results and Evaluation

The minimum calculated Mean Squared Error (MSE) for the test set is: **0.0075**. The below figure demonstrates the results of the model on 6 different test images (one test image per column) after 1, 20, and 44 epochs (first, second, and third row, respectively).


Predicted keypoints vs. ground truth on test images with Direct Regression Model.

# 3 Part 2: Transfer Learning for Keypoint Detection




ResNet-34 Backbone Architecture with Custom Regression Head

## 3.1 Pretrained ResNet Backbone

For this section, we utilized a pretrained ResNet-34 architecture [2] as the backbone for our keypoint detector. ResNet-34 was trained on RGB images to perform 10-class classification of object pictures. Hence, to enable this pretrained model to perform our keypoint classification task, modifications to the input layer and head were necessary. To ensure ResNet-34's compatibility with our single-channel face-image dataset, we replace the input layer to expect the appropriate amount of channels:

Listing 1: ResNet-34 conv1 (Input) Modification

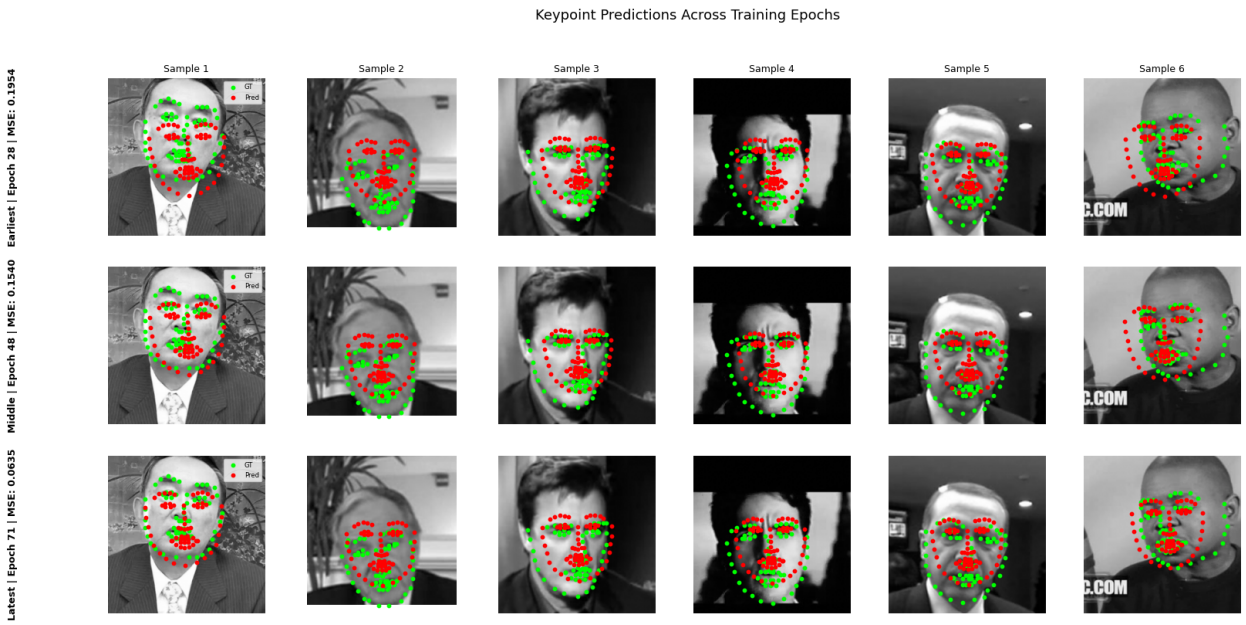
```
model.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)
```

Moreover, in order to ensure the model outputs matched the expected number of keypoints, we replaced the head with a staircased FC layers with ReLU activation and dropout ($p = 0.5$), similar to our head in 2:

Listing 2: ResNet-34 FC Head Modification

```
model.fc = nn.Sequential(
    nn.Linear(num_ftrs, 432),
    nn.ReLU(),
    nn.Dropout(p=dropout_p),
    nn.Linear(432, 216),
    nn.ReLU(),
    nn.Dropout(p=dropout_p),
    nn.Linear(216, 136)
)
```

## 3.2 Performance Comparison



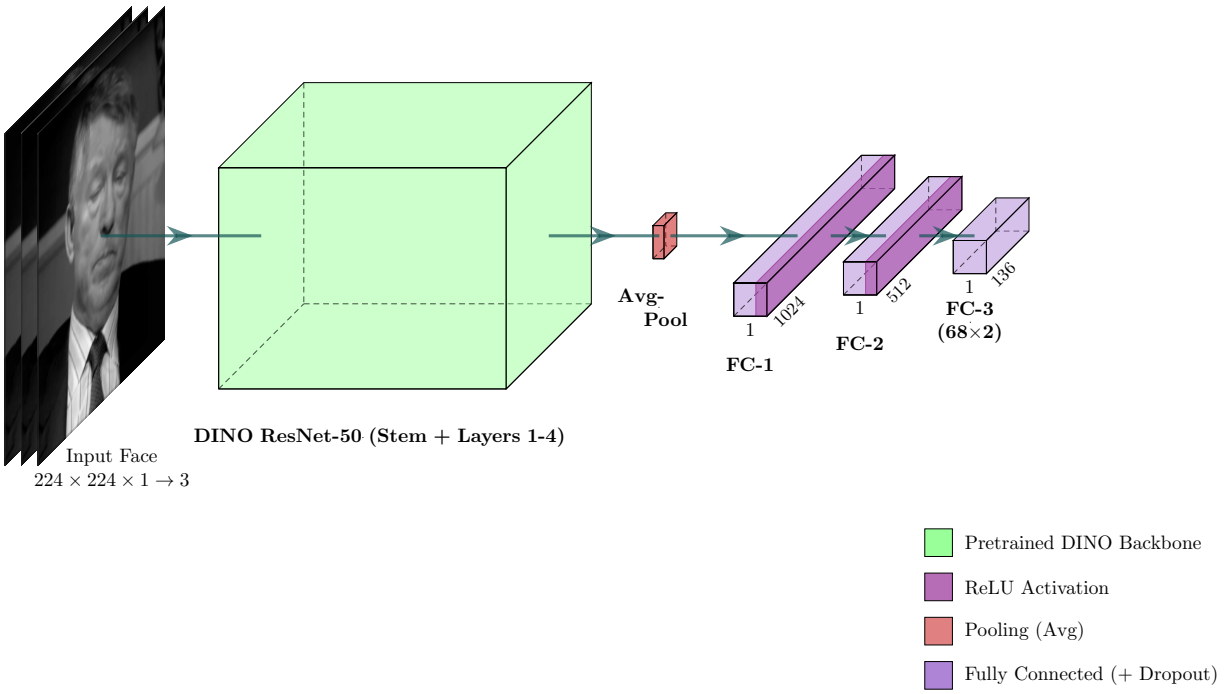

Predicted keypoints vs. ground truth on test images with finetuned ResNet model.

The methodology utilized to train and finetune this model consisted of 3 phases: (Phase 1) freezing the pretrained backbone and training the new layers only (figure 6.5), (Phase 2) freezing all but the last layer of the pretrained backbone (layer 4) and finetuning it with the new layers, and (Phase 3) unfreezing all new and pretrained layers and finetuning the entire model (figure 6.6). Training epochs were 50, 10, and 50 for each phase, respectively. During Phase 1, the hyperparameters used during training include dropout $p = 0.5$,

an initial learning rate of $1 \times 10^{-3}$, and Smooth L1 Loss is used as the loss function. During Phase 2 and Phase 3, the learning rate for the FC head was lowered to $1 \times 10^{-4}$, and pretrained layer learning rates was lowered to $1 \times 10^{-5}$.

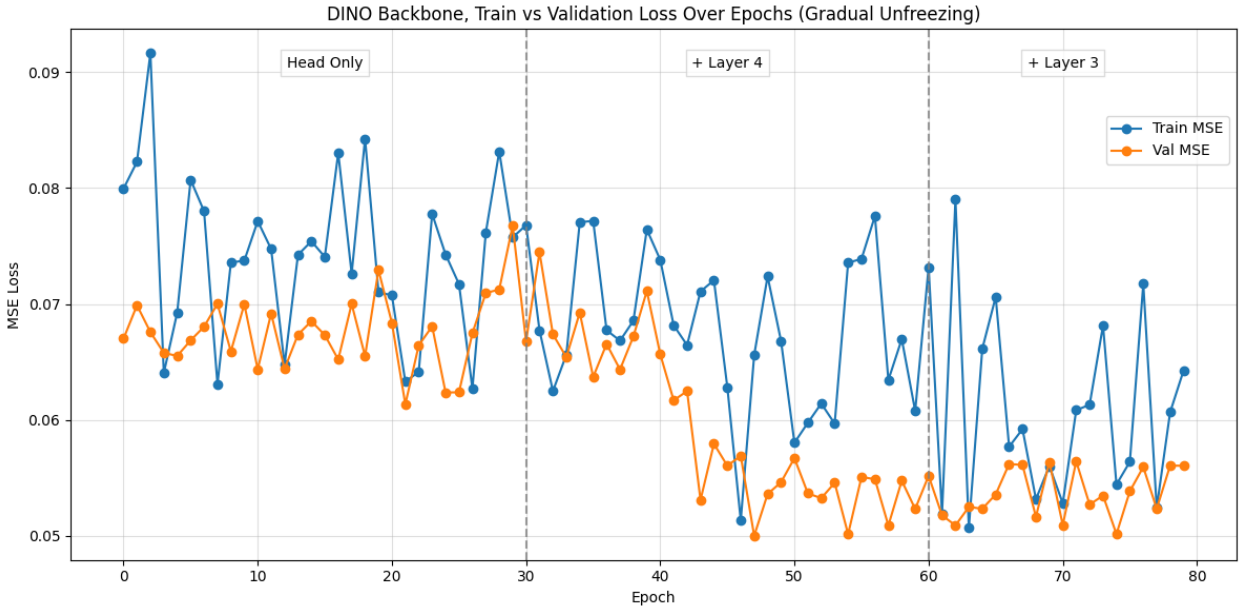
The minimum calculated Mean Squared Error (MSE) for the test set is: **0.0635**. The above 3.2 demonstrates the results of the model on 6 different test images (one test image per column) after 28 (phase 1), 48 (phase 1), and 71 (phase 3) epochs (first, second, and third row, respectively). Overall, despite a collective 100+ epochs of training compared to Direct Coordinate Regression's 50 epochs, the ResNet-34 Backbone architecture has greater MSE by an order of magnitude.

## 3.3 Advanced Pretrained Models (Bonus)




DINO ResNet-50 Backbone Architecture with Grayscale-to-RGB Conversion and Custom Regression Head

The methodology utilized to train and finetune DINO [1] ResNet-50 consisted of 3 phases: (Phase 1) freezing the pretrained backbone and training the new head only, (Phase 2) freezing all but the last layer of the pretrained backbone (layer 4) and finetuning the head along with the new layers, and (Phase 3) unfreezing layers 4, 3, and the head and finetuning further (figure 3.3). Training epochs were 30, 30, and 10 for each phase, respectively. During Phase 1, the hyperparameters used during training include dropout $p = 0.5$, an initial learning rate of $1 \times 10^{-2}$, and Smooth L1 Loss is used as the loss function. During Phase 2 and Phase 3, the learning rate for the FC head was lowered to $1 \times 10^{-4}$, and pretrained layer learning rates was lowered to $1 \times 10^{-5}$.




Training and Validation Loss for the DINO ResNet-50 backed architecture.

The minimum calculated MSE for the test set is: **0.1160**. The below 3.3 demonstrates the results of the model on 6 different test images (one test image per column) after 0 (phase 1), 23 (phase 1), and 43 (phase 3) epochs (first, second, and third row, respectively). Overall, the performance of this model after 80 epochs was worse than our ResNet-34 backed model.


Predicted keypoints vs. ground truth on test images with finetuned DINO ResNet-50.

# 4 Part 3: Heatmap-based Keypoint Detection

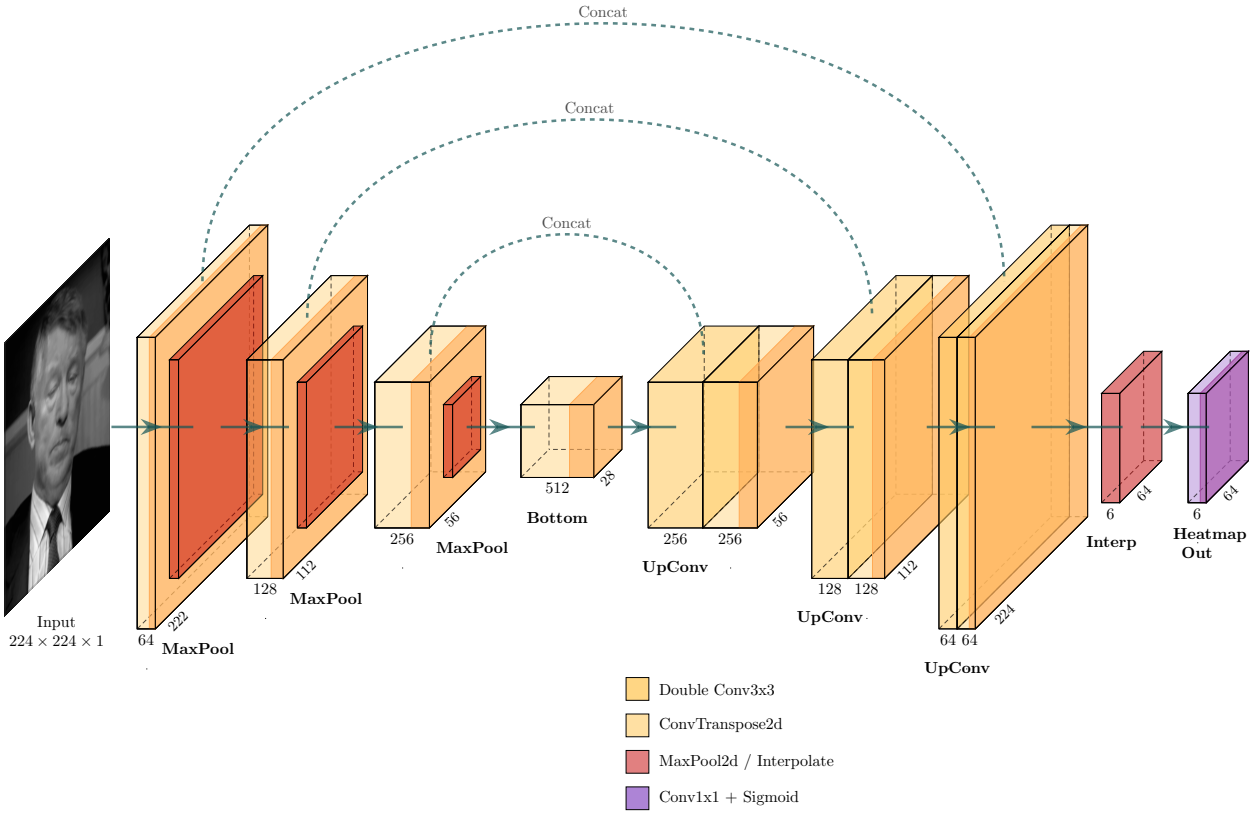
## 4.1 Heatmap Generation

Gaussian heatmaps were generated in `FacialKeypointsHeatmapDataset.generate_heatmaps()` 5 at a resolution of $64 \times 64$, lower than the input image size of $224 \times 224$. For each of the 68 keypoints, the normalized keypoint coordinates (in the range $[-2, 2]$, produced by the `Normalize` transform) were first mapped into the heatmap's pixel space using:

$$x_{\text{scaled}} = x \cdot \frac{H}{4} + \frac{H}{2} \tag{2}$$

where $H = 64$ is the heatmap output size. This maps the $[-2, 2]$ range linearly to $[0, 64]$. A single hot pixel was then placed at the nearest integer coordinate $(x_{\text{int}}, y_{\text{int}})$ in an otherwise zero $64 \times 64$ grid. A Gaussian blur (`scipy.ndimage.gaussian_filter`, $\sigma = 1$) was applied to this hot pixel to produce a soft activation peak, simulating spatial uncertainty around the keypoint location. The resulting heatmap was normalized to $[0, 1]$ by dividing by its maximum value, ensuring consistent scale across all keypoints and samples regardless of peak. At inference time, predicted keypoint coordinates were recovered from the heatmaps by taking the arg max of each $64 \times 64$ channel and mapping back to image space.

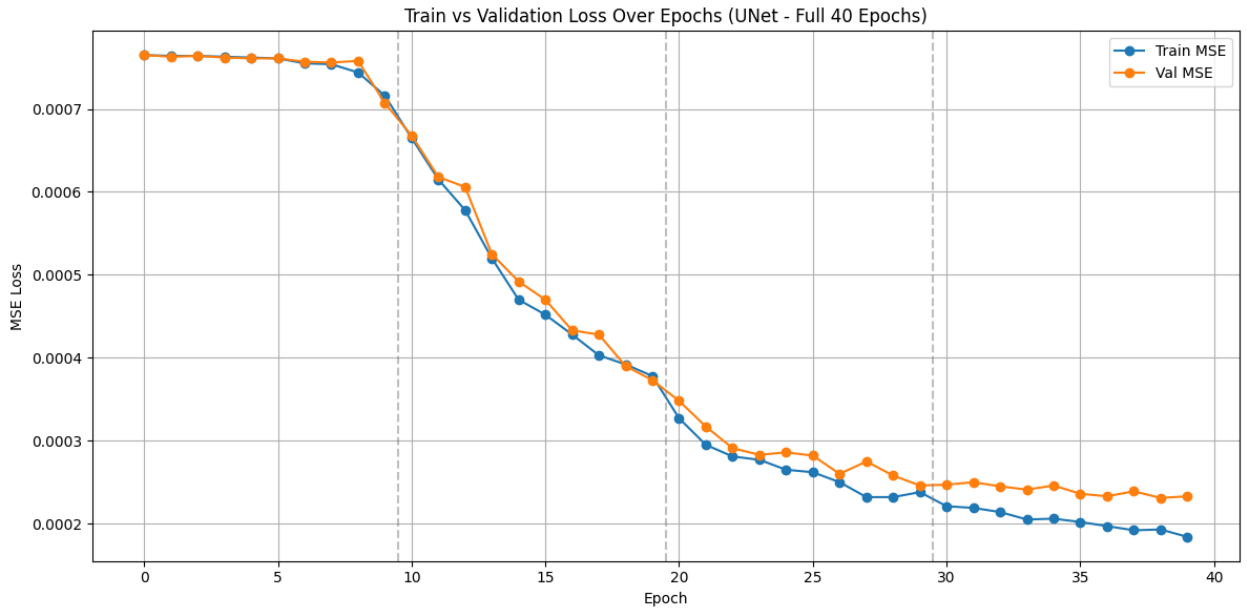
## 4.2 U-Net Heatmap Prediction




Custom U-Net Architecture for $224 \times 224$ to $64 \times 64$ Heatmap Synthesis


Predicted heatmaps/keypoints vs. ground truth on test images with custom UNet.

During training, we use MSE between ground truth and predicted heatmaps for loss, we use a learning rate of $1 \times 10^{-3}$, and train for 40 epochs 4.2.




Training and Validation Loss for the U-Net architecture.

## 4.3 Visualizations and Comparison

The minimum calculated MSE for the test set is: **0.0002**. The figure 4.3 demonstrates the results of the model on four different test images (one test image for every two column). Each row of test images visualizes results after 0, 26, and 38 training epochs, respectively. Overall, the UNet outperforms all other models by an order of magnitude.

# 5 Discussion and Analysis

## 5.1 Strengths and Weaknesses

Evaluating the results across the three approaches, each method exhibited distinct trade-offs.

**Direct Coordinate Regression:** The custom CNN was straightforward to implement and relatively fast to train, achieving a respectable test MSE of 0.0075. However, direct regression forces the network to map highly spatial 2D features into 1D coordinates through dense layers. Because of the reductive nature of this architecture, spatial topography is lost; hence, this is a difficult task for this architecture.

**Transfer Learning (ResNet-34 and DINO ResNet-50):** Surprisingly, the pretrained architectures performed worse than our simple custom CNN, yielding test MSEs of 0.0635 and 0.1160, respectively. While these backbones are incredibly powerful feature extractors, they were originally designed for image-level classification rather than dense pixel-level regression tasks. The aggressive spatial downsampling (e.g., global average pooling) essentially bottlenecks the precise localization information needed for keypoints. Furthermore, there is a notable domain shift between the RGB object-centric datasets these models were pretrained on and our single-channel grayscale facial crops.

**Heatmap-based Detection (U-Net):** The U-Net approach vastly outperformed the other models, achieving a test MSE of 0.0002. By framing the problem as a dense pixel-wise classification/regression task via heatmaps, the architecture retains spatial dimensions throughout. The skip connections directly map high-resolution structural features from the encoder to the decoder, allowing for highly precise localization. The primary weakness of this approach is the increased computational overhead during training and the required post-processing (e.g., $\arg\max$) to convert heatmaps back to discrete $(x, y)$ coordinates during inference.

## 5.2 Challenges Faced

One of the most significant challenges encountered during this assignment involved data scaling and training stability. Initially, we faced a critical issue with the data pipeline involving shifting from static to dynamic keypoint normalization (code snippets 3 and 4). The target keypoints were scaled using a hardcoded formula that failed to properly center or bound the coordinates when applying dynamic transformations, such as a $224 \times 224$ random crop. This misalignment left the network attempting to predict unscaled, large coordinate values, which resulted in massive Mean Squared Error (MSE) penalties and caused the gradients to explode, ultimately yielding a `nan` loss.

To resolve this instability, we implemented a dynamic normalization strategy that calculates the exact center and scale factor based on the specific dimensions of the cropped image. By transforming each coordinate $p$ using the formula $p' = \frac{p - w/2}{w/4}$ (where $w$ is the image width), the target keypoints are consistently centered at zero and strictly bounded to a $[-2.0, 2.0]$ range. This crop-agnostic approach prevents extreme outlier gradients and stabilizes the network's optimization landscape during training.

Another challenge was successfully adapting the pretrained ResNet and DINO models for our regression task. Modifying the input layer to accept single-channel images and stripping the classification heads to output exactly 136 neurons required careful dimension matching. Even after successfully fine-tuning through multiple phased unfreezing steps, mitigating the loss of spatial resolution from the deep pooling layers remained an inherent structural hurdle that our transfer learning models could not easily overcome.

# 6 Appendix: Plots + Code Snippets

## 6.1

Listing 3: Custom Transforms and Normalization

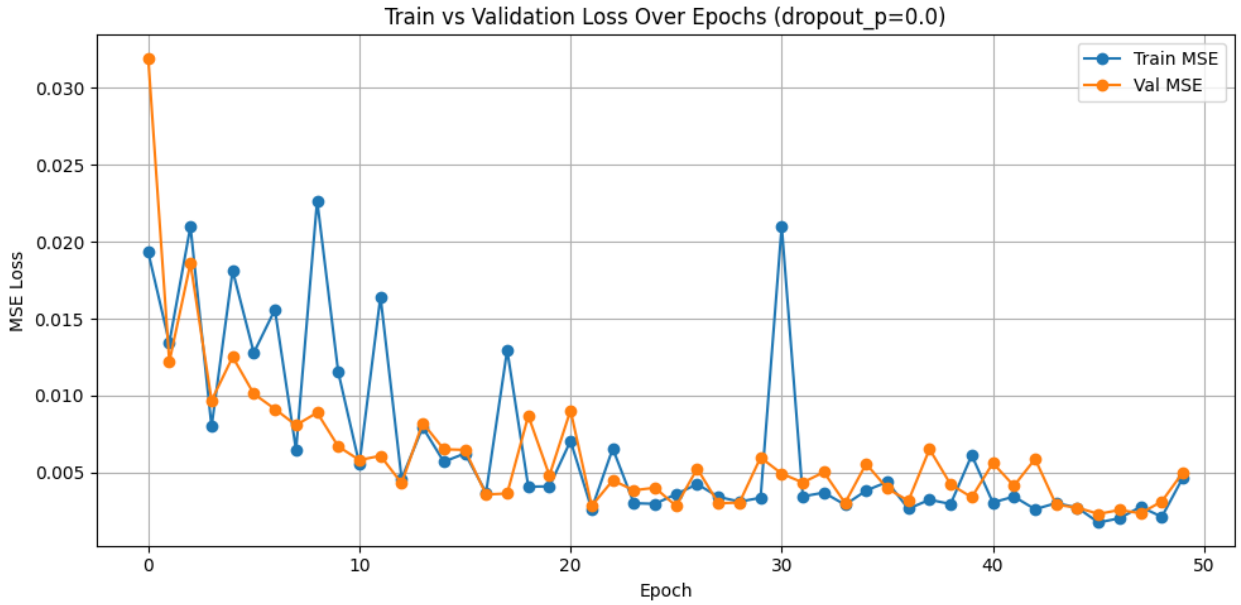
```python
import torch
from torchvision.transforms import Normalize
from custom_transforms import Rescale, RandomCrop, ToTensor

# the transforms we defined in Notebook 1 are in the helper file `
    custom_transforms.py`
from custom_transforms import (
    Rescale,
    RandomCrop,
    Normalize, # <-- Updated!
    ToTensor,
)

# the dataset we created in Notebook 1
from facial_keypoints_dataset import FacialKeypointsDataset

# defining the data_transform using transforms.Compose([all tx's, . , .])
# order matters! i.e. rescaling should come before a smaller crop
data_transform = transforms.Compose(
    [Rescale(250), RandomCrop(224), Normalize(), ToTensor()] # <-- Updated!
)
```

## 6.2

Listing 4: Modification of Normalize Class

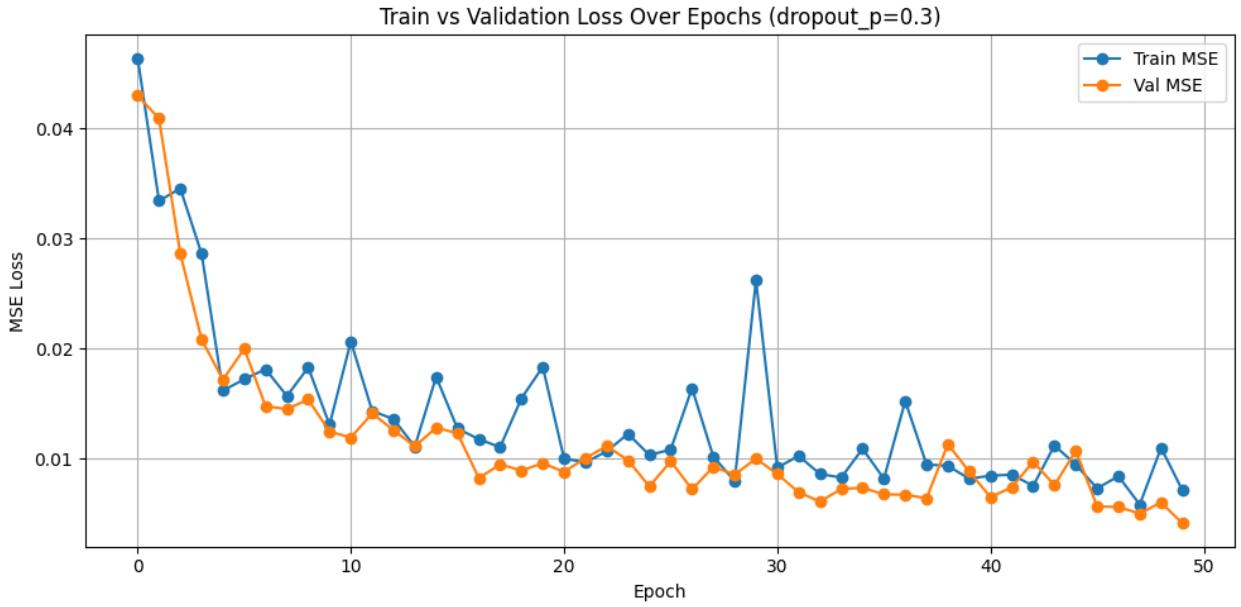
```python
class Normalize(object):
    """Normalize␣the␣color␣range␣to␣[0,1]␣and␣dynamically␣scale␣keypoints"""
    def __call__(self, sample):
        image, key_pts = sample["image"], sample["keypoints"]

        image_copy = np.copy(image)
        key_pts_copy = np.copy(key_pts)

        # Scale image pixels from [0, 255] to [0.0, 1.0]
        image_copy = image_copy / 255.0

        # MODIFICATION: Scale keypoints dynamically based on actual image size
        img_size = image_copy.shape[0]
        key_pts_copy = (key_pts_copy - img_size / 2.0) / (img_size / 4.0)

        return {"image": image_copy, "keypoints": key_pts_copy}
```

## 6.3



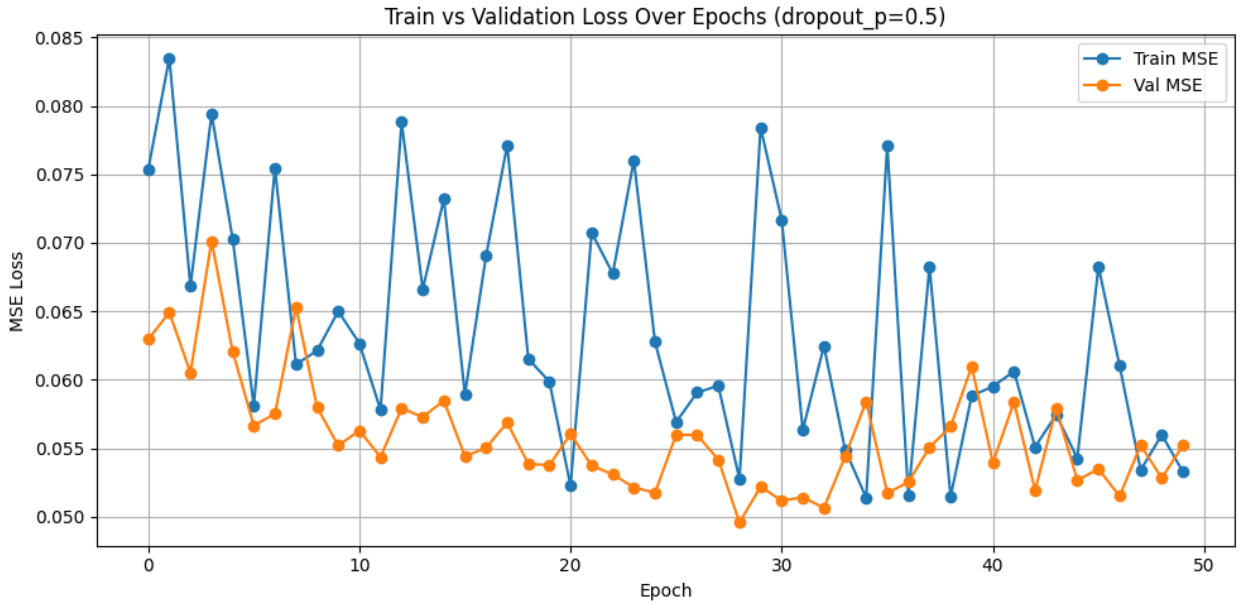

Training and Validation Loss curves for Direct Coordinate Regression ($p = 0.0$).

## 6.4



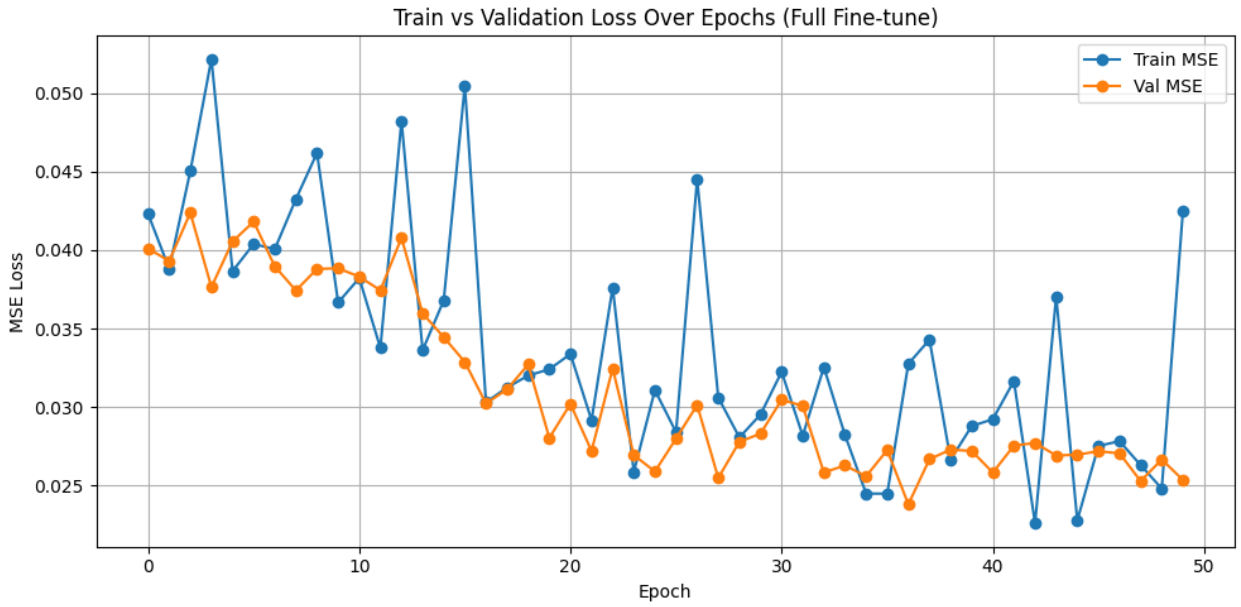

Training and Validation Loss curves for Direct Coordinate Regression ($p = 0.3$).

## 6.5




Phase 1: Training and Validation Loss for the ResNet new layer training.

## 6.6




Phase 3: Training and Validation Loss for the ResNet full finetuning.

### 6.7

Listing 5: Heatmap generation code (modified from starter)

```python
    def generate_heatmaps(self, keypoints):
        # Convert keypoints to numpy if it's a tensor
        if isinstance(keypoints, torch.Tensor):
            keypoints = keypoints.numpy()

        num_keypoints = keypoints.shape[0]
        heatmaps = np.zeros((num_keypoints, self.output_size, self.output_size), dtype=np.float32)

        # FIXED MATH: Map [-2, 2] coordinates directly to the heatmap size (e.g., 64x64)
        scale_factor = self.output_size / 4.0
        center_offset = self.output_size / 2.0
        keypoints_scaled = keypoints * scale_factor + center_offset

        #keypoints_scaled = keypoints * 50 + 100

        # Generate a heatmap for each keypoint
        for i in range(num_keypoints):
            # Get the scaled coordinates
            x, y = keypoints_scaled[i]

            # Skip if keypoint is invalid
            if np.isnan(x) or np.isnan(y):
                continue

            # Convert to int for indexing
            x_int, y_int = max(0, min(self.output_size-1, int(x))), max(0, min(self.output_size-1, int(y)))

            # Create a single hot pixel
            heatmap = np.zeros((self.output_size, self.output_size), dtype=np.float32)
            heatmap[y_int, x_int] = 1.0

            # Apply gaussian filter to create a soft heatmap
            heatmap = gaussian_filter(heatmap, sigma=self.sigma)
            heatmap = heatmap/heatmap.max()

            # Normalize heatmap to 0-1 range
            if heatmap.max() > 0:
                heatmap = heatmap / heatmap.max()

            heatmaps[i] = heatmap

        return torch.from_numpy(heatmaps)
```

### 6.8 Complete Python Notebook

# Setup

## Download Data

```
# Fetch data
# !mkdir data
# !wget -P data/ https://s3.amazonaws.com/video.udacity-data.com/topher/2018/May/5aea1b91_train-test-data/train-test-data.zip
# !unzip -n data/train-test-data.zip -d data
```

Show hidden output

## Setup Environment

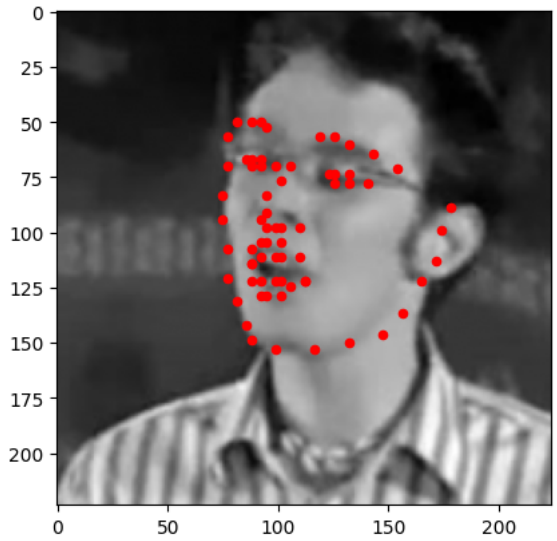
```
# No need if you are using colab

# !pip install matplotlib~=3.5.2
# !pip install scikit-image==0.19.2
# !pip install torch~=1.8.1
# !pip install torchvision~=0.9.1
# !pip install numpy~=1.21.6
# !pip install pillow~=9.1.1
# !pip install tqdm~=4.64.0
# !pip install jupyter==1.0.0
# !pip install opencv-python==4.6.0.66
# !pip install pandas==1.3.5
```

## Visualize the Data

```
# importing the required libraries
import glob
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import cv2
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import transforms, utils
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
import numpy as np
from torch.utils.data import Dataset, DataLoader
from torch.utils.data.sampler import SubsetRandomSampler
from torch.optim import lr_scheduler
import torch.optim as optim
from tqdm import tqdm


# the transforms we defined in Notebook 1 are in the helper file `custom_transforms.py`
from custom_transforms import (
    Rescale,
    RandomCrop,
    Normalize, # <-- Updated!
    ToTensor,
)

# the dataset we created in Notebook 1
from facial_keypoints_dataset import FacialKeypointsDataset

# defining the data_transform using transforms.Compose([all tx's, . , .])
# order matters! i.e. rescaling should come before a smaller crop
data_transform = transforms.Compose(
    [Rescale(250), RandomCrop(224), Normalize(), ToTensor()] # <-- Updated!
)
```

```
training_keypoints_csv_path = os.path.join("data", "training_frames_keypoints.csv")
training_data_dir = os.path.join("data", "training")
test_keypoints_csv_path = os.path.join("data", "test_frames_keypoints.csv")
test_data_dir = os.path.join("data", "test")


# create the transformed dataset
transformed_dataset = FacialKeypointsDataset(
    csv_file=training_keypoints_csv_path,
    root_dir=training_data_dir,
    transform=data_transform,
)

# load training data in batches
batch_size = 16
train_loader = DataLoader(
    transformed_dataset, batch_size=batch_size, shuffle=True, num_workers=4
)

# creating the test dataset
test_dataset = FacialKeypointsDataset(
    csv_file=test_keypoints_csv_path,
    root_dir=test_data_dir,
    transform=data_transform
)

# loading test data in batches
batch_size = 16
test_loader = DataLoader(
    test_dataset, batch_size=batch_size, shuffle=True, num_workers=4
)

example_idx = 0
for i, data in enumerate(test_loader):
    sample = data
    image = sample['image'][example_idx]

    keypoints = sample['keypoints'][example_idx]

    img_size = image.shape[-1]  # 224

    plt.imshow(image.numpy().transpose(1, 2, 0), cmap='gray')
    #now that keypoints are normalized dynamically based on image size,
    #the way we reverse the normalization needs to be dynamic as well
    plt.scatter(
        keypoints[:, 0] * (img_size / 4.0) + (img_size / 2.0),
        keypoints[:, 1] * (img_size / 4.0) + (img_size / 2.0),
        c='r', s=20
    )
    plt.show()
    break
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:424: UserWarning: This DataLoader will create 4 worker pr
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:424: UserWarning: This DataLoader will create 4 worker pr
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:432: UserWarning: This DataLoader will create 4 worker pr
  self.check_worker_number_rationality()
```




## Training

### Part 1: Direct Coordinate Regression

```
# PyTorch models inherit from torch.nn.Module

#CNN component requirements:
# · Convolutional layers with appropriate kernel sizes, strides, and padding

# · Batch normalization for stable training
#Q: Is batch normalization different from the normalization that is applied to the data already?
#A: This is different from data normalizaion.
# It recomputes batch norm during training because the distribution of the data can radically shift
# as we move through the layers, this helps keep it centered throughout (which I suppose is useful for stability)

# · Activation functions (ReLU, ELU, etc.)
# These are useful becaues they introduce non-linearities. Without these, the network mathmatically collapses into a single
# linear transformation (Ax + b); activation functions force more complicated behavior

# · Pooling layers to reduce spatial dimensions; Q: what are "pooling layers"?
# A: they reduce spatial dimension lol. Lke from 255x255xD to, say, 16x16xD.
# For example average-pooling neighboring pixel values together.

# · Dropout for regularization; Q1: What is dropout regularization?
# A: Randomly drop information in neurons to help avoid overfitting
# Q2: How would you do this? During test-time w/probability p, output the 0 tensor instead of layer tranformation?
# A: x = self.dropout(x); NOTE: remember to switch between train()/eval() to auto use all neurons during deployment

# . Fully connected layers that output 136 values (68 keypoints × 2 coordinates)

class FacialKeypointCNN(nn.Module):

    def __init__(self, drop_out_p=0.3):
        super(FacialKeypointCNN, self).__init__()
        # Inputs:
          # Image shape:  torch.Size([1, 224, 224])
        # Outputs:
          # Keypoints shape:  torch.Size([68, 2])

        # convolutional layers
        # (input_channels, output_channels, kernel_size)
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=32,
```

```
                                kernel_size=3, stride=1, padding=1)
        self.conv2 = nn.Conv2d(in_channels=32, out_channels=64,
                                kernel_size=3, stride=1, padding=1)
        self.conv3 = nn.Conv2d(in_channels=64, out_channels=128,
                                kernel_size=3, stride=1, padding=1)
        self.conv4 = nn.Conv2d(in_channels=128, out_channels=256,
                                kernel_size=3, stride=1, padding=1)
        self.conv5 = nn.Conv2d(in_channels=256, out_channels=512,
                                kernel_size=3, stride=1, padding=1)
        self.conv6 = nn.Conv2d(512, 128, kernel_size=1) #bottleneck layer

        # pooling function
        self.pool = nn.MaxPool2d(2, 2) #(size, stride)

        # dropout layer
        # every neuron has a 30% chance of being zerod out druing a training step
        self.dropout = nn.Dropout(drop_out_p)

        # batch norm
        self.bn1 = nn.BatchNorm2d(32)
        self.bn2 = nn.BatchNorm2d(64)
        self.bn3 = nn.BatchNorm2d(128)
        self.bn4 = nn.BatchNorm2d(256)
        self.bn5 = nn.BatchNorm2d(512)
        self.bn6 = nn.BatchNorm2d(128)


        # Start of fully-connected layers
        self.fc1 = nn.Linear(128 * 7 * 7, 1024)
        self.fc2 = nn.Linear(1024, 512)
        self.fc3 = nn.Linear(512, 136)

    def forward(self, x):
        # NOTE: there's always an implicit batch dimension (e.g. (D, 1, 28, 28))

        # kernel_size = 3; padding = 0; padding = 1; stride = 1; 2x2 average(?) pooling
        # output_dim = ((input_dim - kernel_size + 2*padding) / stride) + 1

        # layer1: (1, 224, 224) -> (32, 224, 224) -> (32, 112, 112); where 224 = (224 - 3 + 2*1) + 1
        x = self.pool(F.relu(self.bn1(self.conv1(x))))

        # layer2: (32, 112, 112) -> (64, 112, 112) -> (64, 56, 56); where 112 = (112 - 3 + 2*1) + 1
        x = self.pool(F.relu(self.bn2(self.conv2(x))))

        # layer3: (64, 56, 56) -> (128, 56, 56) -> (128, 28, 28); where 56 = (56 - 3 + 2*1) + 1
        x = self.pool(F.relu(self.bn3(self.conv3(x))))

        # layer4: (128, 28, 28) -> (256, 28, 28) -> (256, 14, 14); where 28 = (28 - 3 + 2*1) + 1
        x = self.pool(F.relu(self.bn4(self.conv4(x))))

        # layer5: (256, 14, 14) -> (512, 14, 14) -> (512, 7, 7); where 14 = (14 - 3 + 2*1) + 1
        x = self.pool(F.relu(self.bn5(self.conv5(x))))

        #layer6: (512, 7, 7) -> (128, 7, 7)
        x = F.relu(self.bn6(self.conv6(x)))


        # (flatten step): (128, 7, 7) -> (128 * 7 * 7) (i.e. batch size by number of entries in x)
        x = x.view(-1, 128 * 7 * 7)

        # (128 * 7 * 7) -> (1024)
        x = self.dropout(F.relu(self.fc1(x)))

        # (1024) -> (512)
        x = self.dropout(F.relu(self.fc2(x)))

        # (512) -> (136)
        x = self.fc3(x)

        return x.view(-1, 68, 2) # return as (68, 2) tensor
```

```
loss_fn = nn.SmoothL1Loss()  # or nn.MSELoss()
```

```
# This will use the GPU if available, otherwise it falls back to CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
p = 0.3 #0.0, 0.3, or 0.5

model = FacialKeypointCNN(drop_out_p=p)
model = model.to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

```
Using device: cuda
```

```
# install if needed
#!pip install torchinfo torchviz --break-system-packages -q
```

```
# from torchinfo import summary
# from torchviz import make_dot

# # --- Option 1: torchinfo table summary ---
# # shows layer names, output shapes, and parameter counts
# summary(model, input_size=(1, 1, 224, 224), device=device)

# # --- Option 2: torchviz computation graph ---
# dummy_input = torch.randn(1, 1, 224, 224).to(device)
# output = model(dummy_input)
# dot = make_dot(output, params=dict(model.named_parameters()))
# dot.format = 'png'
# dot.render('model_architecture')  # saves model_architecture.png
# dot.view()                        # opens it if a viewer is available
```

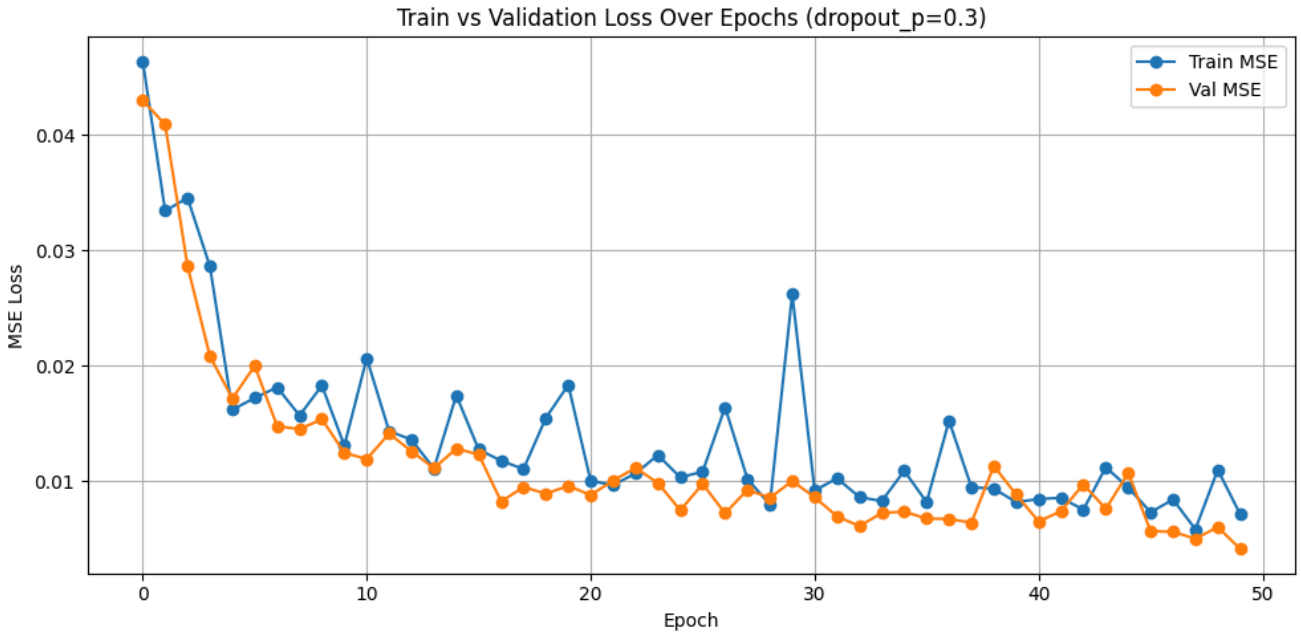
```
from tqdm.auto import tqdm
def train_one_epoch(epoch_index, tb_writer, optimizer):
    running_loss = 0.
    last_loss = 0.

    # Wrap the loader with tqdm
    # Here, we use enumerate(training_loader) instead of
    # iter(training_loader) so that we can track the batch
    pbar = tqdm(enumerate(train_loader), total=len(train_loader), desc=f"Epoch {epoch_index + 1} Training")

    # index and do some intra-epoch reporting
    for i, data in pbar:
        # Every data instance is an input + label pair
        sample = data
        inputs, labels = sample['image'], sample['keypoints']
        inputs = inputs.to(device)
        labels = labels.to(device)

        # Zero your gradients for every batch!
        optimizer.zero_grad()

        # Make predictions for this batch
        outputs = model(inputs)

        # Compute the loss and its gradients
        loss = loss_fn(outputs, labels)
        loss.backward()

        # Adjust learning weights
        optimizer.step()

        # Gather data and report
        running_loss += loss.item()

        # Update the progress bar with the current loss
        pbar.set_postfix({'loss': f"{loss.item():.4f}"})

        if i % 10 == 9:
            last_loss = running_loss / 10 # loss per batch
            #print('  batch {} loss: {}'.format(i + 1, last_loss))
            tb_x = epoch_index * len(train_loader) + i + 1
            tb_writer.add_scalar('Loss/train', last_loss, tb_x)
            running_loss = 0.

    return last_loss
```

```
# TODO: Training a simple CNN
from datetime import datetime
```

```
from torch.utils.tensorboard import SummaryWriter
import glob

# --- Resume from latest checkpoint if available ---
checkpoint_files = glob.glob('model_*')
if checkpoint_files:
    latest_checkpoint = max(checkpoint_files, key=lambda x: int(x.split('_')[-1]))
    print(f"Resuming from checkpoint: {latest_checkpoint}")
    model.load_state_dict(torch.load(latest_checkpoint, map_location=device))
    # parse epoch number from filename (format: model_{timestamp}_{epoch})
    epoch_number = int(latest_checkpoint.split('_')[-1]) + 1
    timestamp = '_'.join(latest_checkpoint.split('_')[1:3])  # preserve original timestamp
    print(f"Resuming from epoch {epoch_number}")
else:
    print("No checkpoint found, starting from scratch.")
    epoch_number = 0
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')

writer = SummaryWriter('runs/fashion_trainer_{}'.format(timestamp))

EPOCHS = 50

best_vloss = 1_000_000.

val_losses = []
train_losses = []

for epoch in range(EPOCHS):
    print('EPOCH {}:'.format(epoch_number + 1))

    # Make sure gradient tracking is on, and do a pass over the data
    model.train(True)
    avg_loss = train_one_epoch(epoch_number, writer, optimizer)


    running_vloss = 0.0
    # Set the model to evaluation mode, disabling dropout and using population
    # statistics for batch normalization.
    model.eval()

    # Disable gradient computation and reduce memory consumption.
    with torch.no_grad():
        # Wrap the validation loader
        val_pbar = tqdm(enumerate(test_loader), total=len(test_loader), desc="Validation")

        for i, vdata in val_pbar:
            vinputs, vlabels = vdata['image'], vdata['keypoints']
            vinputs = vinputs.to(device)
            vlabels = vlabels.to(device)

            voutputs = model(vinputs)
            vloss = loss_fn(voutputs, vlabels)
            running_vloss += vloss.item() # Use .item() to keep memory clean

    avg_vloss = running_vloss / (i + 1)
    print('LOSS train {} valid {}'.format(avg_loss, avg_vloss))
    train_losses.append(avg_loss)
    val_losses.append(avg_vloss)

    # Log the running loss averaged per batch
    # for both training and validation
    writer.add_scalars('Training vs. Validation Loss',
                    { 'Training' : avg_loss, 'Validation' : avg_vloss },
                    epoch_number + 1)
    writer.flush()

    # Track best performance, and save the model's state
    if avg_vloss < best_vloss:
        best_vloss = avg_vloss
        model_path = 'model_{}_{}'.format(timestamp, epoch_number)
        torch.save(model.state_dict(), model_path)

    epoch_number += 1
```

Show hidden output

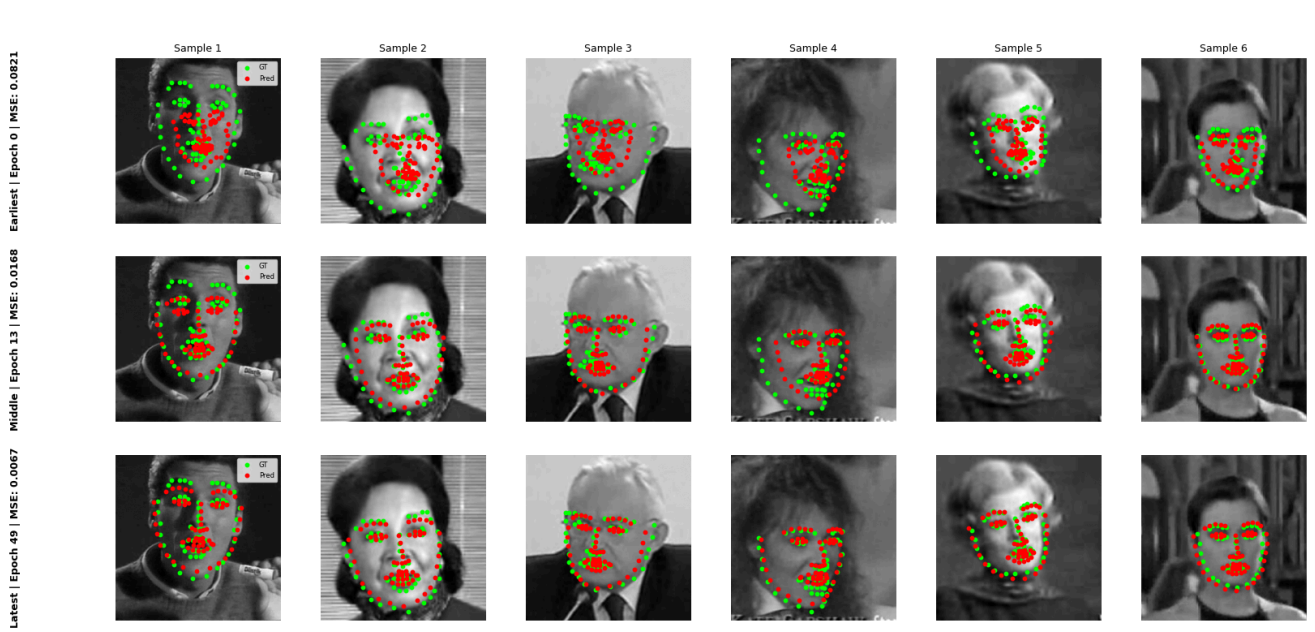
```
plt.figure(figsize=(10, 5))
plt.plot(range(EPOCHS), train_losses, marker='o', label='Train MSE')
plt.plot(range(EPOCHS), val_losses,   marker='o', label='Val MSE')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Train vs Validation Loss Over Epochs (dropout_p=0.3)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```




```
import glob
import re

# --- Find and sort checkpoints by epoch number ---
checkpoint_files = glob.glob('model_*')
if not checkpoint_files:
    raise FileNotFoundError("No checkpoint files found.")

checkpoint_files = sorted(checkpoint_files, key=lambda x: int(x.split('_')[-1]))
print(f"Found checkpoints: {checkpoint_files}")

# Pick earliest, middle, latest
indices = [0, len(checkpoint_files) // 2, -1]
selected_checkpoints = [checkpoint_files[i] for i in indices]
checkpoint_labels = ['Earliest', 'Middle', 'Latest']
print(f"Selected: {list(zip(checkpoint_labels, selected_checkpoints))}")

mse_fn = nn.MSELoss()

# --- Grab a fixed batch from test_loader ---
#TODO: uncomment, when done comparing
fixed_batch = next(iter(test_loader))
fixed_inputs = fixed_batch['image'].to(device)
fixed_labels = fixed_batch['keypoints'].to(device)
num_examples = 6
img_size = fixed_inputs.shape[-1]

# --- Plot: rows = checkpoints, cols = examples ---
fig, axes = plt.subplots(3, num_examples, figsize=(18, 9))

for row, (ckpt_path, ckpt_label) in enumerate(zip(selected_checkpoints, checkpoint_labels)):
    epoch_num = ckpt_path.split('_')[-1]
    model.load_state_dict(torch.load(ckpt_path, map_location=device))
    model.eval()

    with torch.no_grad():
        outputs = model(fixed_inputs)
        batch_mse = mse_fn(outputs, fixed_labels).item()
```

```
    outputs_cpu = outputs.cpu()
    fixed_labels_cpu = fixed_labels.cpu()

    for col in range(num_examples):
        img = fixed_inputs[col].cpu().numpy().transpose(1, 2, 0).squeeze()
        gt_kps   = fixed_labels_cpu[col].numpy() * (img_size / 4.0) + (img_size / 2.0)
        pred_kps = outputs_cpu[col].numpy()       * (img_size / 4.0) + (img_size / 2.0)

        ax = axes[row, col]
        ax.imshow(img, cmap='gray')
        ax.scatter(gt_kps[:, 0],   gt_kps[:, 1],   c='lime', s=10, label='GT')
        ax.scatter(pred_kps[:, 0], pred_kps[:, 1], c='red',  s=10, label='Pred')
        ax.axis('off')

        if col == 0:
            ax.legend(loc='upper right', fontsize=6)
        if row == 0:
            ax.set_title(f"Sample {col + 1}", fontsize=9)

    # Add row label as a figure-level text annotation instead of ylabel
    fig.text(
        0.01,                                        # x: near left edge
        axes[row, 0].get_position().y0 + axes[row, 0].get_position().height / 2,  # y: row center
        f"{ckpt_label} | Epoch {epoch_num} | MSE: {batch_mse:.4f}",
        va='center', ha='left', fontsize=9, fontweight='bold',
        rotation=90
    )

plt.suptitle("Keypoint Predictions Across Training Epochs", fontsize=13)
plt.subplots_adjust(left=0.08)  # make room for the row labels on the left
plt.show()
```

```
Found checkpoints: ['model_20260225_070355_0', 'model_20260225_070355_1', 'model_20260225_070355_2', 'model_20260225_070355_3',
Selected: [('Earliest', 'model_20260225_070355_0'), ('Middle', 'model_20260225_070355_13'), ('Latest', 'model_20260225_070355_49
```




## Part 2: Transfer Learning for Keypoint Detection

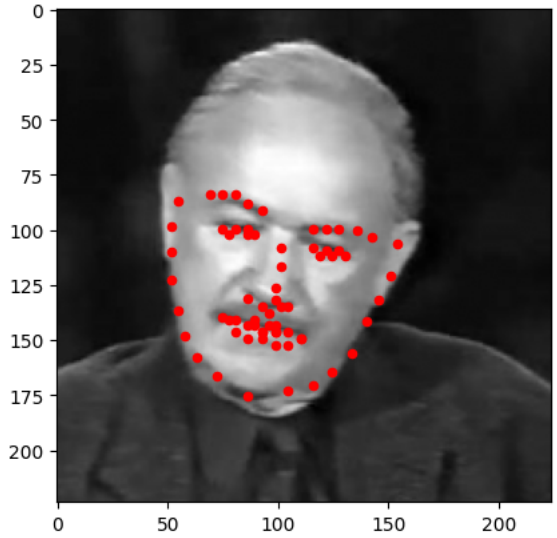
```
# Fetch data
!mkdir data
!wget -P data/ https://s3.amazonaws.com/video.udacity-data.com/topher/2018/May/5aea1b91_train-test-data/train-test-data.zip
!unzip -n data/train-test-data.zip -d data
```

Show hidden output

```
# importing the required libraries
import glob
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import cv2
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import transforms, utils
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
import numpy as np
from torch.utils.data import Dataset, DataLoader
from torch.utils.data.sampler import SubsetRandomSampler
from torch.optim import lr_scheduler
import torch.optim as optim
from tqdm import tqdm


# the transforms we defined in Notebook 1 are in the helper file `custom_transforms.py`
from custom_transforms import (
    Rescale,
    RandomCrop,
    Normalize,
    ToTensor,
)

# the dataset we created in Notebook 1
from facial_keypoints_dataset import FacialKeypointsDataset

# defining the data_transform using transforms.Compose([all tx's, . , .])
# order matters! i.e. rescaling should come before a smaller crop
data_transform = transforms.Compose(
    [Rescale(250), RandomCrop(224), Normalize(), ToTensor()] # <-- Updated!
)

training_keypoints_csv_path = os.path.join("data", "training_frames_keypoints.csv")
training_data_dir = os.path.join("data", "training")
test_keypoints_csv_path = os.path.join("data", "test_frames_keypoints.csv")
test_data_dir = os.path.join("data", "test")


# create the transformed dataset
transformed_dataset = FacialKeypointsDataset(
    csv_file=training_keypoints_csv_path,
    root_dir=training_data_dir,
    transform=data_transform,
)

# load training data in batches
batch_size = 16
train_loader = DataLoader(
    transformed_dataset, batch_size=batch_size, shuffle=True, num_workers=4
)

# creating the test dataset
test_dataset = FacialKeypointsDataset(
    csv_file=test_keypoints_csv_path,
    root_dir=test_data_dir,
    transform=data_transform
)

# loading test data in batches
batch_size = 16
test_loader = DataLoader(
    test_dataset, batch_size=batch_size, shuffle=True, num_workers=4
)
```

```
example_idx = 0
for i, data in enumerate(test_loader):
    sample = data
    image = sample['image'][example_idx]

    keypoints = sample['keypoints'][example_idx]

    img_size = image.shape[-1]  # 224

    plt.imshow(image.numpy().transpose(1, 2, 0), cmap='gray')
    #now that keypoints are normalized dynamically based on image size,
    #the way we reverse the normalization needs to be dynamic as well
    plt.scatter(
        keypoints[:, 0] * (img_size / 4.0) + (img_size / 2.0),
        keypoints[:, 1] * (img_size / 4.0) + (img_size / 2.0),
        c='r', s=20
    )
    plt.show()
    break
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:424: UserWarning: This DataLoader will create 4 worker pr
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:424: UserWarning: This DataLoader will create 4 worker pr
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:432: UserWarning: This DataLoader will create 4 worker pr
  self.check_worker_number_rationality()
```




```
# TODO: Pretrained ResNet backbone
import torch
import torch.nn as nn
import torchvision.models as models

# Load the pre-trained model
model = models.resnet34(weights='IMAGENET1K_V1')
#print(model)

# Freeze all existing parameters
for param in model.parameters():
    param.requires_grad = False

# TODO: Replace model first layer to accept single-channel input
# Save the old weights
old_weights = model.conv1.weight.data # Shape is [64, 3, 7, 7]

# Create the new 1-channel layer
model.conv1 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)

# If I wanted to preserve the weights of the old head
# Sum the old weights across the RGB channel (dimension 1) and assign to new layer
# New shape becomes [64, 1, 7, 7]
#model.conv1.weight.data = torch.sum(old_weights, dim=1, keepdim=True) / 3
```

```
# TODO: Replace output layer so that it doesn't take the square root of PI
# Get the number of input features for the final fully connected layer
num_ftrs = model.fc.in_features  # For ResNet-34, this is 512

# Replace the 'fc' layer with a new one
dropout_p = 0.5 #0.0, 0.3, or 0.5
model.fc = nn.Sequential(
    nn.Linear(num_ftrs, 432), #fc1
    nn.ReLU(),
    nn.Dropout(p=dropout_p),
    nn.Linear(432, 216), #fc2
    nn.ReLU(),
    nn.Dropout(p=dropout_p),
    nn.Linear(216, 136), #fc3
    nn.Unflatten(dim=1, unflattened_size=(68, 2))
)

print(model)
```

Show hidden output

```
loss_fn = nn.SmoothL1Loss()  # or nn.MSELoss()
```

```
# This will use the GPU if available, otherwise it falls back to CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

model = model.to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

```
Using device: cuda
```

```
# @title
from tqdm.auto import tqdm
def train_one_epoch(epoch_index, tb_writer, optimizer):
    running_loss = 0.
    last_loss = 0.

    # Wrap the loader with tqdm
    # Here, we use enumerate(training_loader) instead of
    # iter(training_loader) so that we can track the batch
    pbar = tqdm(enumerate(train_loader), total=len(train_loader), desc=f"Epoch {epoch_index + 1} Training")

    # index and do some intra-epoch reporting
    for i, data in pbar:
        # Every data instance is an input + label pair
        sample = data
        inputs, labels = sample['image'], sample['keypoints']
        inputs = inputs.to(device)
        labels = labels.to(device)

        # Zero your gradients for every batch!
        optimizer.zero_grad()

        # Make predictions for this batch
        outputs = model(inputs)

        # Compute the loss and its gradients
        loss = loss_fn(outputs, labels)
        loss.backward()

        # Adjust learning weights
        optimizer.step()

        # Gather data and report
        running_loss += loss.item()

        # Update the progress bar with the current loss
        pbar.set_postfix({'loss': f"{loss.item():.4f}"})

        if i % 10 == 9:
            last_loss = running_loss / 10 # loss per batch
            #print('  batch {} loss: {}'.format(i + 1, last_loss))
            tb_x = epoch_index * len(train_loader) + i + 1
            tb_writer.add_scalar('Loss/train', last_loss, tb_x)
```

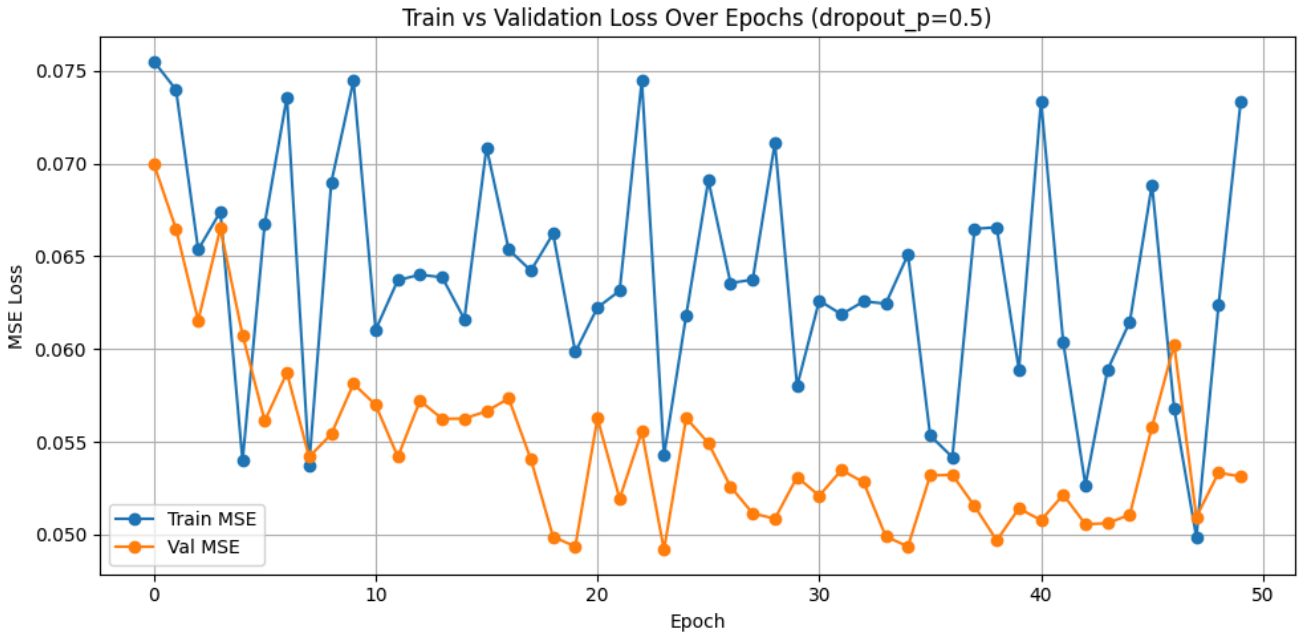
```
            running_loss = 0.

    return last_loss
```

```
# TODO: Training CNN w/IMAGENET backbone
from datetime import datetime
from torch.utils.tensorboard import SummaryWriter
import glob

# --- Resume from latest checkpoint if available ---
checkpoint_files = glob.glob('model_*')
if checkpoint_files:
    latest_checkpoint = max(checkpoint_files, key=lambda x: int(x.split('_')[-1]))
    print(f"Resuming from checkpoint: {latest_checkpoint}")
    model.load_state_dict(torch.load(latest_checkpoint, map_location=device))
    # parse epoch number from filename (format: model_{timestamp}_{epoch})
    epoch_number = int(latest_checkpoint.split('_')[-1]) + 1
    timestamp = '_'.join(latest_checkpoint.split('_')[1:3])  # preserve original timestamp
    print(f"Resuming from epoch {epoch_number}")
else:
    print("No checkpoint found, starting from scratch.")
    epoch_number = 0
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')

writer = SummaryWriter('runs/fashion_trainer_{}'.format(timestamp))

EPOCHS = 50

best_vloss = 1_000_000.

val_losses = []
train_losses = []

for epoch in range(EPOCHS):
    print('EPOCH {}:'.format(epoch_number + 1))

    # Make sure gradient tracking is on, and do a pass over the data
    model.train(True)
    avg_loss = train_one_epoch(epoch_number, writer, optimizer)


    running_vloss = 0.0
    # Set the model to evaluation mode, disabling dropout and using population
    # statistics for batch normalization.
    model.eval()

    # Disable gradient computation and reduce memory consumption.
    with torch.no_grad():
        # Wrap the validation loader
        val_pbar = tqdm(enumerate(test_loader), total=len(test_loader), desc="Validation")

        for i, vdata in val_pbar:
            vinputs, vlabels = vdata['image'], vdata['keypoints']
            vinputs = vinputs.to(device)
            vlabels = vlabels.to(device)

            voutputs = model(vinputs)
            vloss = loss_fn(voutputs, vlabels)
            running_vloss += vloss.item() # Use .item() to keep memory clean

    avg_vloss = running_vloss / (i + 1)
    print('LOSS train {} valid {}'.format(avg_loss, avg_vloss))
    train_losses.append(avg_loss)
    val_losses.append(avg_vloss)

    # Log the running loss averaged per batch
    # for both training and validation
    writer.add_scalars('Training vs. Validation Loss',
                    { 'Training' : avg_loss, 'Validation' : avg_vloss },
                    epoch_number + 1)
    writer.flush()

    # Track best performance, and save the model's state
    if avg_vloss < best_vloss:
        best_vloss = avg_vloss
        model_path = 'model_{}_{}'.format(timestamp, epoch_number)
```

```
            torch.save(model.state_dict(), model_path)

        epoch_number += 1
```

Show hidden output

```
plt.figure(figsize=(10, 5))
plt.plot(range(EPOCHS), train_losses, marker='o', label='Train MSE')
plt.plot(range(EPOCHS), val_losses,   marker='o', label='Val MSE')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Train vs Validation Loss Over Epochs (dropout_p=0.5)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```




Show code

```
Found checkpoints: ['model_20260226_020409_0', 'model_20260226_020409_1', 'model_20260226_020409_11', 'model_20260226_020409_15'
Selected: [('Earliest', 'model_20260226_020409_0'), ('Middle', 'model_20260226_020409_15'), ('Latest', 'model_20260226_020409_40
```


```
# After phase 1 training converges, unfreeze layer4 only
for param in model.layer4.parameters():
    param.requires_grad = True

# Use a much lower LR than phase 1
optimizer = torch.optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 1e-5},
    {'params': model.fc.parameters(),     'lr': 1e-4},  # head can tolerate slightly higher LR
])
```

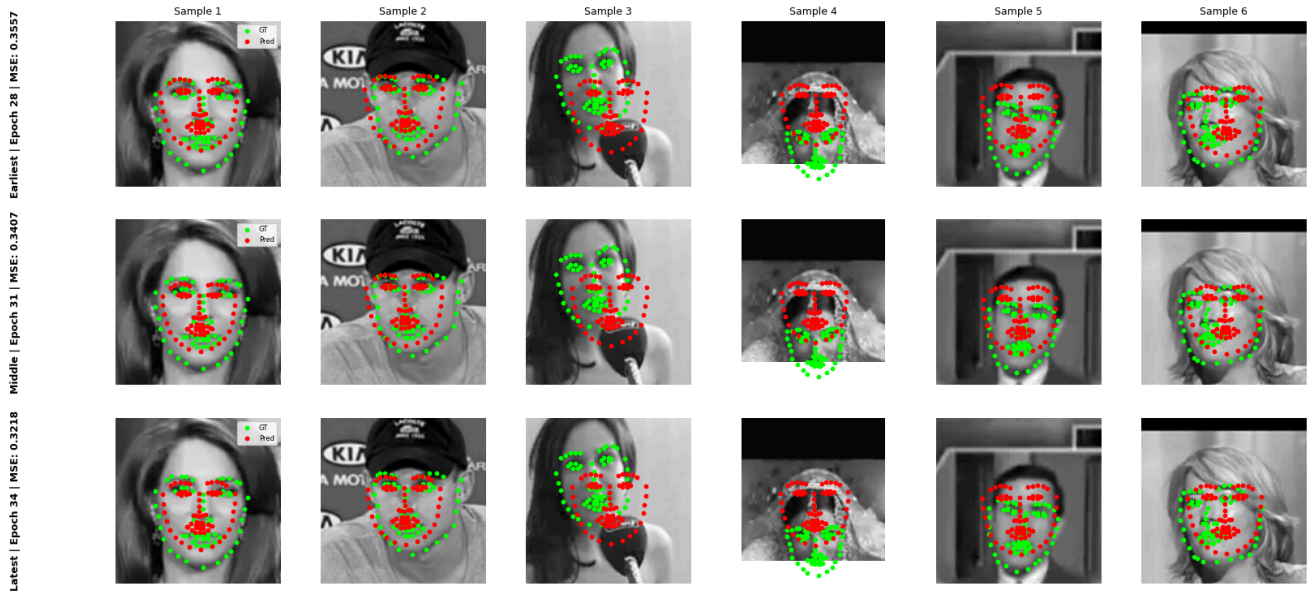
```
# TODO: Phase 2: unphreasing the second to last layer
# --- Resume from latest checkpoint if available ---
checkpoint_files = glob.glob('model_*')
if checkpoint_files:
    latest_checkpoint = max(checkpoint_files, key=lambda x: int(x.split('_')[-1]))
    print(f"Resuming from checkpoint: {latest_checkpoint}")
    model.load_state_dict(torch.load(latest_checkpoint, map_location=device))
    # parse epoch number from filename (format: model_{timestamp}_{epoch})
    epoch_number = int(latest_checkpoint.split('_')[-1]) + 1
    timestamp = '_'.join(latest_checkpoint.split('_')[1:3])  # preserve original timestamp
    print(f"Resuming from epoch {epoch_number}")
else:
    print("No checkpoint found, starting from scratch.")
    epoch_number = 0
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')

writer = SummaryWriter('runs/fashion_trainer_{}'.format(timestamp))

EPOCHS = 50

best_vloss = 1_000_000.

val_losses = []
train_losses = []

for epoch in range(EPOCHS):
    print('EPOCH {}:'.format(epoch_number + 1))
```

```
# Make sure gradient tracking is on, and do a pass over the data
model.train(True)
avg_loss = train_one_epoch(epoch_number, writer, optimizer)


running_vloss = 0.0
# Set the model to evaluation mode, disabling dropout and using population
# statistics for batch normalization.
model.eval()

# Disable gradient computation and reduce memory consumption.
with torch.no_grad():
    # Wrap the validation loader
    val_pbar = tqdm(enumerate(test_loader), total=len(test_loader), desc="Validation")

    for i, vdata in val_pbar:
        vinputs, vlabels = vdata['image'], vdata['keypoints']
        vinputs = vinputs.to(device)
        vlabels = vlabels.to(device)

        voutputs = model(vinputs)
        vloss = loss_fn(voutputs, vlabels)
        running_vloss += vloss.item() # Use .item() to keep memory clean

avg_vloss = running_vloss / (i + 1)
print('LOSS train {} valid {}'.format(avg_loss, avg_vloss))
train_losses.append(avg_loss)
val_losses.append(avg_vloss)

# Log the running loss averaged per batch
# for both training and validation
writer.add_scalars('Training vs. Validation Loss',
                { 'Training' : avg_loss, 'Validation' : avg_vloss },
                epoch_number + 1)
writer.flush()

# Track best performance, and save the model's state
if avg_vloss < best_vloss:
    best_vloss = avg_vloss
    model_path = 'model_{}_{}'.format(timestamp, epoch_number)
    torch.save(model.state_dict(), model_path)

epoch_number += 1
```

Show hidden output

Show code

```
Found checkpoints: ['model_20260226_023754_28', 'model_20260226_023754_29', 'model_20260226_023754_30', 'model_20260226_023754_3
Selected: [('Earliest', 'model_20260226_023754_28'), ('Middle', 'model_20260226_023754_31'), ('Latest', 'model_20260226_023754_3
```



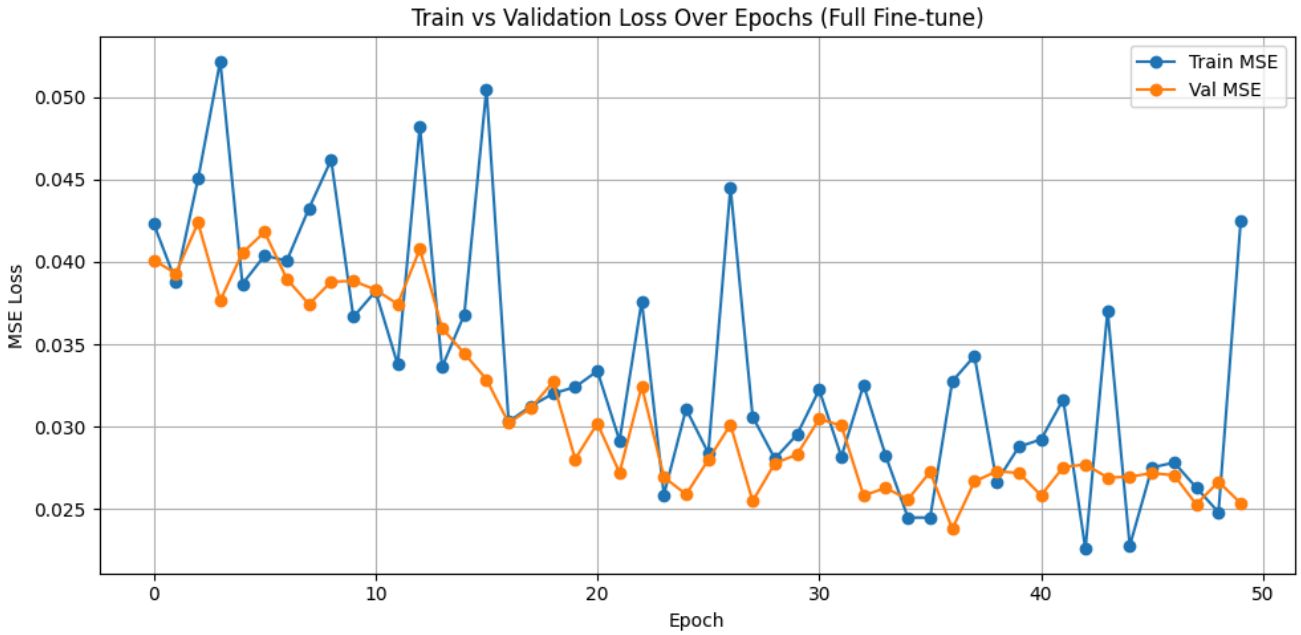

```
# After phase 3 training converges, unfreeze all layers (1-4) only
for param in model.layer4.parameters():
    param.requires_grad = True
for param in model.layer3.parameters():
    param.requires_grad = True
for param in model.layer2.parameters():
    param.requires_grad = True
for param in model.layer1.parameters():
    param.requires_grad = True

# Use a much lower LR than phase 1
optimizer = torch.optim.Adam([
    {'params': model.layer1.parameters(), 'lr': 1e-5},
    {'params': model.layer2.parameters(), 'lr': 1e-5},
    {'params': model.layer3.parameters(), 'lr': 1e-5},
    {'params': model.layer4.parameters(), 'lr': 1e-5},
    {'params': model.fc.parameters(),     'lr': 1e-4},  # head can tolerate slightly higher LR
])
```

```
# TODO: Phase3: unphreasing the entire model
# --- Resume from latest checkpoint if available ---
checkpoint_files = glob.glob('model_*')
if checkpoint_files:
    latest_checkpoint = max(checkpoint_files, key=lambda x: int(x.split('_')[-1]))
    print(f"Resuming from checkpoint: {latest_checkpoint}")
    model.load_state_dict(torch.load(latest_checkpoint, map_location=device))
    # parse epoch number from filename (format: model_{timestamp}_{epoch})
    epoch_number = int(latest_checkpoint.split('_')[-1]) + 1
    timestamp = '_'.join(latest_checkpoint.split('_')[1:3])  # preserve original timestamp
    print(f"Resuming from epoch {epoch_number}")
else:
    print("No checkpoint found, starting from scratch.")
    epoch_number = 0
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')

writer = SummaryWriter('runs/fashion_trainer_{}'.format(timestamp))

EPOCHS = 50
```

```
best_vloss = 1_000_000.

val_losses = []
train_losses = []

for epoch in range(EPOCHS):
    print('EPOCH {}:'.format(epoch_number + 1))

    # Make sure gradient tracking is on, and do a pass over the data
    model.train(True)
    avg_loss = train_one_epoch(epoch_number, writer, optimizer)


    running_vloss = 0.0
    # Set the model to evaluation mode, disabling dropout and using population
    # statistics for batch normalization.
    model.eval()

    # Disable gradient computation and reduce memory consumption.
    with torch.no_grad():
        # Wrap the validation loader
        val_pbar = tqdm(enumerate(test_loader), total=len(test_loader), desc="Validation")

        for i, vdata in val_pbar:
            vinputs, vlabels = vdata['image'], vdata['keypoints']
            vinputs = vinputs.to(device)
            vlabels = vlabels.to(device)

            voutputs = model(vinputs)
            vloss = loss_fn(voutputs, vlabels)
            running_vloss += vloss.item() # Use .item() to keep memory clean

    avg_vloss = running_vloss / (i + 1)
    print('LOSS train {} valid {}'.format(avg_loss, avg_vloss))
    train_losses.append(avg_loss)
    val_losses.append(avg_vloss)

    # Log the running loss averaged per batch
    # for both training and validation
    writer.add_scalars('Training vs. Validation Loss',
                    { 'Training' : avg_loss, 'Validation' : avg_vloss },
                    epoch_number + 1)
    writer.flush()

    # Track best performance, and save the model's state
    if avg_vloss < best_vloss:
        best_vloss = avg_vloss
        model_path = 'model_{}_{}'.format(timestamp, epoch_number)
        torch.save(model.state_dict(), model_path)

    epoch_number += 1
```

Show hidden output

```
plt.figure(figsize=(10, 5))
plt.plot(range(EPOCHS), train_losses, marker='o', label='Train MSE')
plt.plot(range(EPOCHS), val_losses,   marker='o', label='Val MSE')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Train vs Validation Loss Over Epochs (Full Fine-tune)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```




Show code

```
Found checkpoints: ['model_20260226_023754_28', 'model_20260226_023754_29', 'model_20260226_023754_30', 'model_20260226_023754_3
Selected: [('Earliest', 'model_20260226_023754_28'), ('Middle', 'model_20260226_023754_48'), ('Latest', 'model_20260226_023754_7
```


```
# TODO: Pretrained ResNet backbone
import torch
import torch.nn as nn
import torchvision.models as models

# Load the pre-trained DINO
resnet50 = torch.hub.load('facebookresearch/dino:main', 'dino_resnet50')
#print(resnet50)
```

```
Using cache found in /root/.cache/torch/hub/facebookresearch_dino_main
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `No
  warnings.warn(msg)
```

```
# Freeze all existing parameters
for param in resnet50.parameters():
    param.requires_grad = False

# TODO: Convert single-channnel data to RGB before feeding into resent50

# TODO: Replace output layer so that it doesn't take the square root of PI
# Get the number of input features for the final fully connected layer
num_ftrs = 2048  # For ResNet-50, this is 2048

# Replace the 'fc' layer with a new one
dropout_p = 0.5 #0.0, 0.3, or 0.5
resnet50.fc = nn.Sequential(
    nn.Linear(num_ftrs, 1024), #fc1
    nn.ReLU(),
    nn.Dropout(p=dropout_p),
    nn.Linear(1024, 512), #fc2
    nn.ReLU(),
    nn.Dropout(p=dropout_p),
    nn.Linear(512, 136), #fc3
    nn.Unflatten(dim=1, unflattened_size=(68, 2))
)
#print(resnet50)
```

```
loss_fn = nn.SmoothL1Loss()  # or nn.MSELoss()
```

```
# This will use the GPU if available, otherwise it falls back to CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

resnet50 = resnet50.to(device)
optimizer = torch.optim.Adam(resnet50.parameters(), lr=1e-2)
```

```
Using device: cuda
```

Show code

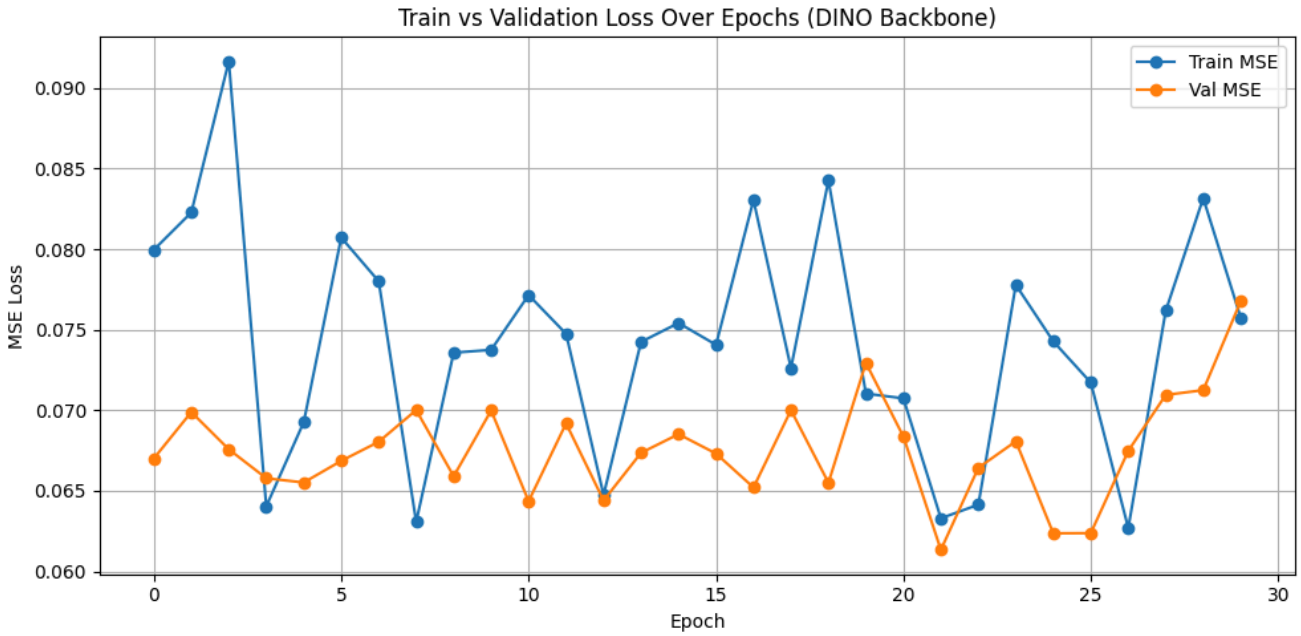
Show code

```
Resuming from checkpoint: model_20260227_035648_10
Resuming from epoch 11
EPOCH 12:
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:432: UserWarning: This DataLoader will create 4 worker p
  self.check_worker_number_rationality()
```

Epoch 12 Training: 100% 217/217 [00:15<00:00, 15.49it/s, loss=0.0705]

Show code

LOSS train 0.07405966865757149 valid 0.0673088644820878

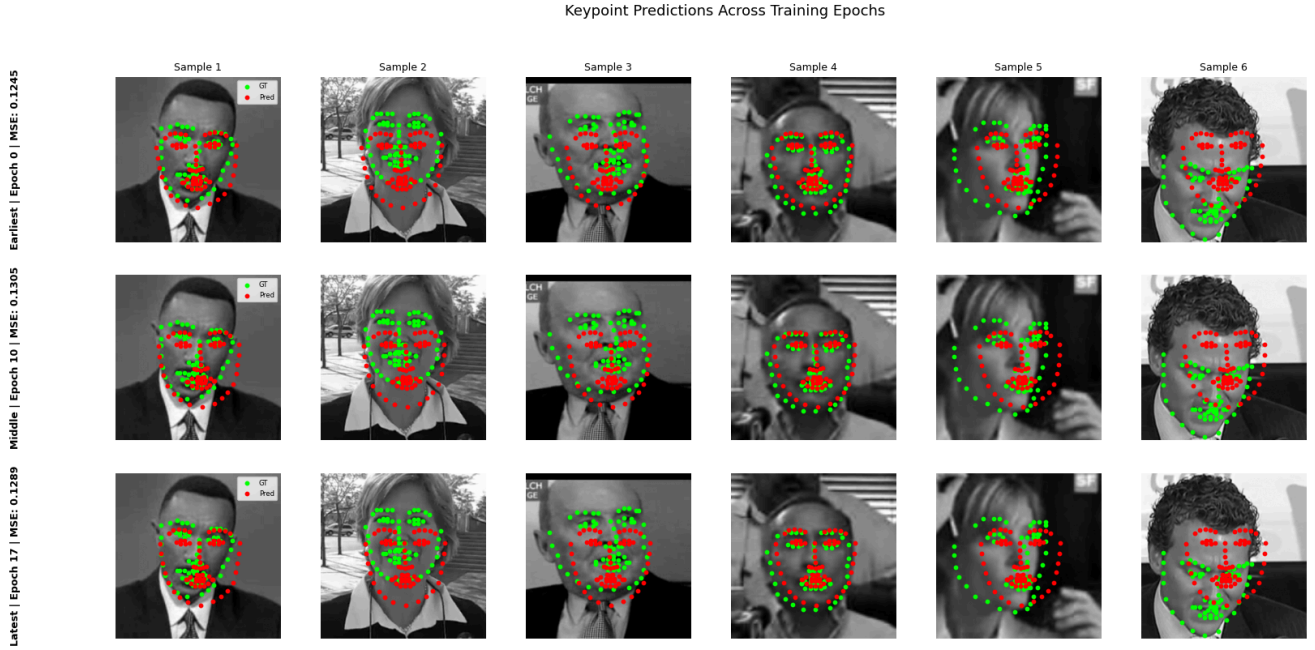



EPOCH 18:

Show code

Validation: 100% 49/49 [00:03<00:00, 18.18it/s]

Found checkpoints: ['model_20260227_035648_0', 'model_20260227_035648_3', 'model_20260227_035648_4', 'model_20260227_035648_10', 'model_20260227_035648_17']
Selected [('Earliest', 'model_20260227_035648_0'), ('Middle', 'model_20260227_035648_10'), ('Latest', 'model_20260227_035648_17')]




EPOCH 24:

Epoch 24 Training: 100% 217/217 [00:15<00:00, 14.51it/s, loss=0.0597]

Validation: 100% 49/49 [00:03<00:00, 18.06it/s]

LOSS train 0.07617316532968109 valid 0.07094732062198503

EPOCH 25:

Epoch 25 Training: 100% 217/217 [00:14<00:00, 16.65it/s, loss=0.1438]

Validation: 100% 49/49 [00:03<00:00, 18.29it/s]

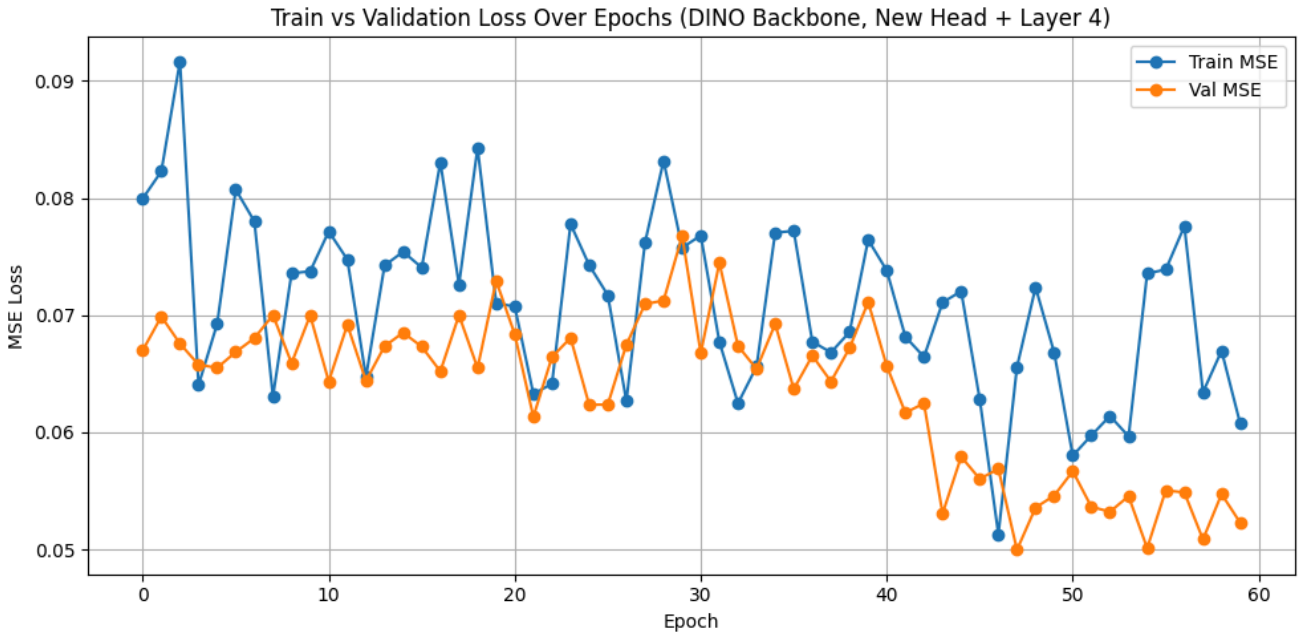
```
# After phase 3 training converges, unfreeze JUST layer 4
#FREEZE layer 4 + head when entering training
for param in resnet50.parameters(): #freezes all others
  param.requires_grad = False
for param in resnet50.layer4.parameters():
  param.requires_grad = True
for param in resnet50.fc.parameters():
  param.requires_grad = True

# Use a much lower LR than phase 1
optimizer = torch.optim.Adam([
    {'params': resnet50.layer4.parameters(), 'lr': 1e-5},
    {'params': resnet50.fc.parameters(),     'lr': 1e-4},  # head can tolerate slightly higher LR
])
```

```
# @title
# TODO: Training w/IMAGENET-50 DINO backbone
from datetime import datetime
from torch.utils.tensorboard import SummaryWriter
import glob

# --- Resume from latest checkpoint if available ---
checkpoint_files = glob.glob('model_*')
if checkpoint_files:
    latest_checkpoint = max(checkpoint_files, key=lambda x: int(x.split('_')[-1]))
    print(f"Resuming from checkpoint: {latest_checkpoint}")
    resnet50.load_state_dict(torch.load(latest_checkpoint, map_location=device))
    # parse epoch number from filename (format: model_{timestamp}_{epoch})
    epoch_number = int(latest_checkpoint.split('_')[-1]) + 1
    timestamp = '_'.join(latest_checkpoint.split('_')[1:3])  # preserve original timestamp
    print(f"Resuming from epoch {epoch_number}")
else:
    print("No checkpoint found, starting from scratch.")
    epoch_number = 0
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')

writer = SummaryWriter('runs/fashion_trainer_{}'.format(timestamp))

EPOCHS = 30

best_vloss = 1_000_000.

# val_losses = []
# train_losses = []

for epoch in range(EPOCHS):
    print('EPOCH {}:'.format(epoch_number + 1))

    # Make sure gradient tracking is on, and do a pass over the data
    resnet50.train(True)
    #REFREEZE layer 4 + head when entering training
    for param in resnet50.parameters(): #freezes all others
      param.requires_grad = False
    for param in resnet50.layer4.parameters():
      param.requires_grad = True
    for param in resnet50.fc.parameters():
      param.requires_grad = True

    avg_loss = train_one_epoch(epoch_number, writer, optimizer)


    running_vloss = 0.0
    # Set the model to evaluation mode, disabling dropout and using population
    # statistics for batch normalization.
    resnet50.eval()

    # Disable gradient computation and reduce memory consumption.
    with torch.no_grad():
        # Wrap the validation loader
        val_pbar = tqdm(enumerate(test_loader), total=len(test_loader), desc="Validation")

        for i, vdata in val_pbar:
            vinputs, vlabels = vdata['image'], vdata['keypoints']
            vinputs = vinputs.repeat(1, 3, 1, 1)  # (B, 1, 224, 224) -> (B, 3, 224, 224)
            vinputs = vinputs.to(device)
            vlabels = vlabels.to(device)
```

```
            voutputs = resnet50(vinputs)
            vloss = loss_fn(voutputs, vlabels)
            running_vloss += vloss.item() # Use .item() to keep memory clean

    avg_vloss = running_vloss / (i + 1)
    print('LOSS train {} valid {}'.format(avg_loss, avg_vloss))
    train_losses.append(avg_loss)
    val_losses.append(avg_vloss)

    # Log the running loss averaged per batch
    # for both training and validation
    writer.add_scalars('Training vs. Validation Loss',
                    { 'Training' : avg_loss, 'Validation' : avg_vloss },
                    epoch_number + 1)
    writer.flush()

    # Track best performance, and save the model's state
    if avg_vloss < best_vloss:
        best_vloss = avg_vloss
        model_path = 'model_{}_{}'.format(timestamp, epoch_number)
        torch.save(resnet50.state_dict(), model_path)

    epoch_number += 1
```

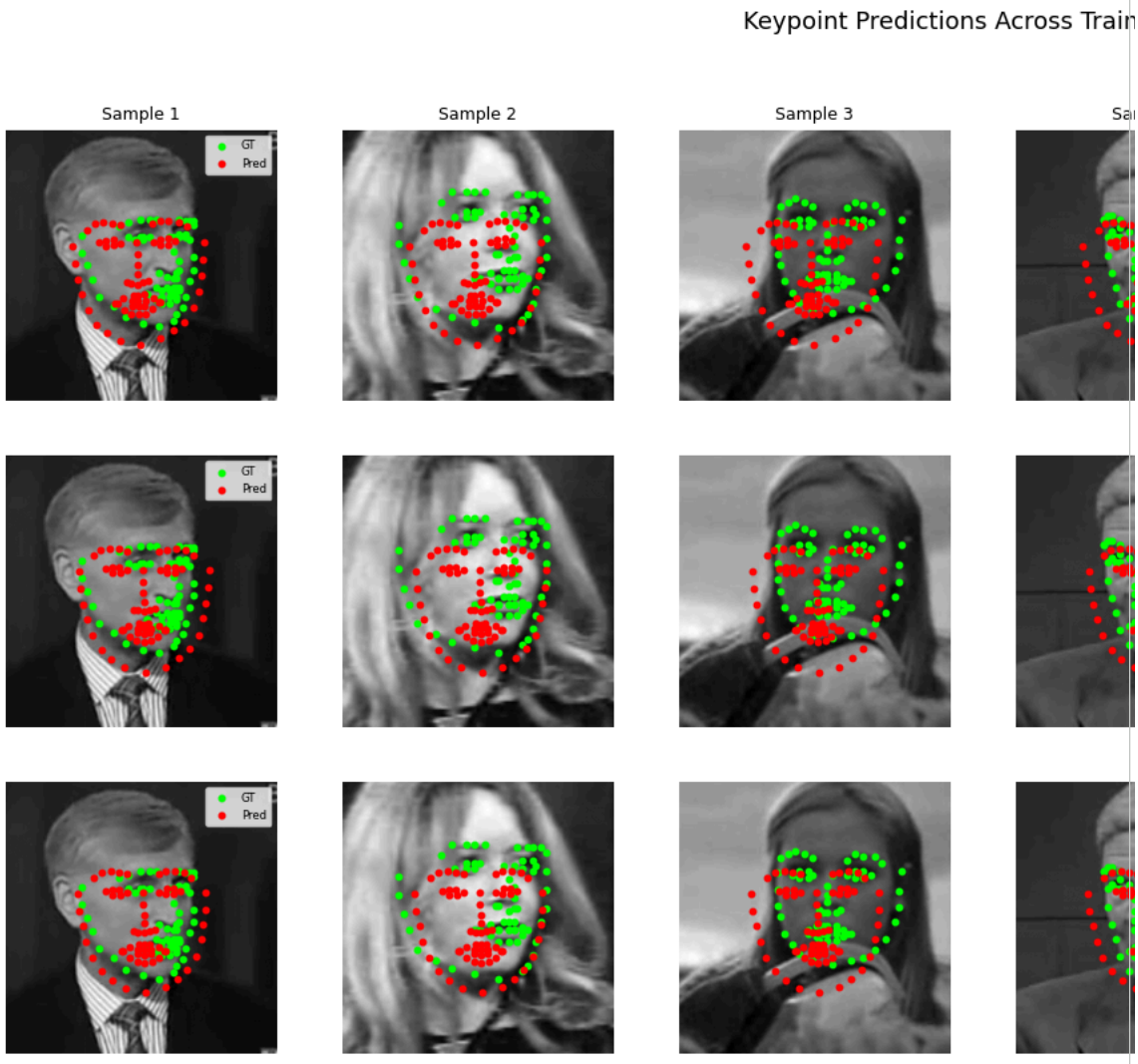
Show hidden output

```
# @title
plt.figure(figsize=(10, 5))
epochs_range = range(len(train_losses))
plt.plot(range(epochs_range), train_losses, marker='o', label='Train MSE')
plt.plot(range(epochs_range), val_losses,   marker='o', label='Val MSE')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Train vs Validation Loss Over Epochs (DINO Backbone, New Head + Layer 4)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```




Show code

```
heckpoints: ['model_20260227_035648_0', 'model_20260227_035648_3', 'model_20260227_035648_4', 'model_20260227_035648_10',
d: [('Earliest', 'model_20260227_035648_0'), ('Middle', 'model_20260227_035648_17'), ('Latest', 'model_20260227_035648_35
```



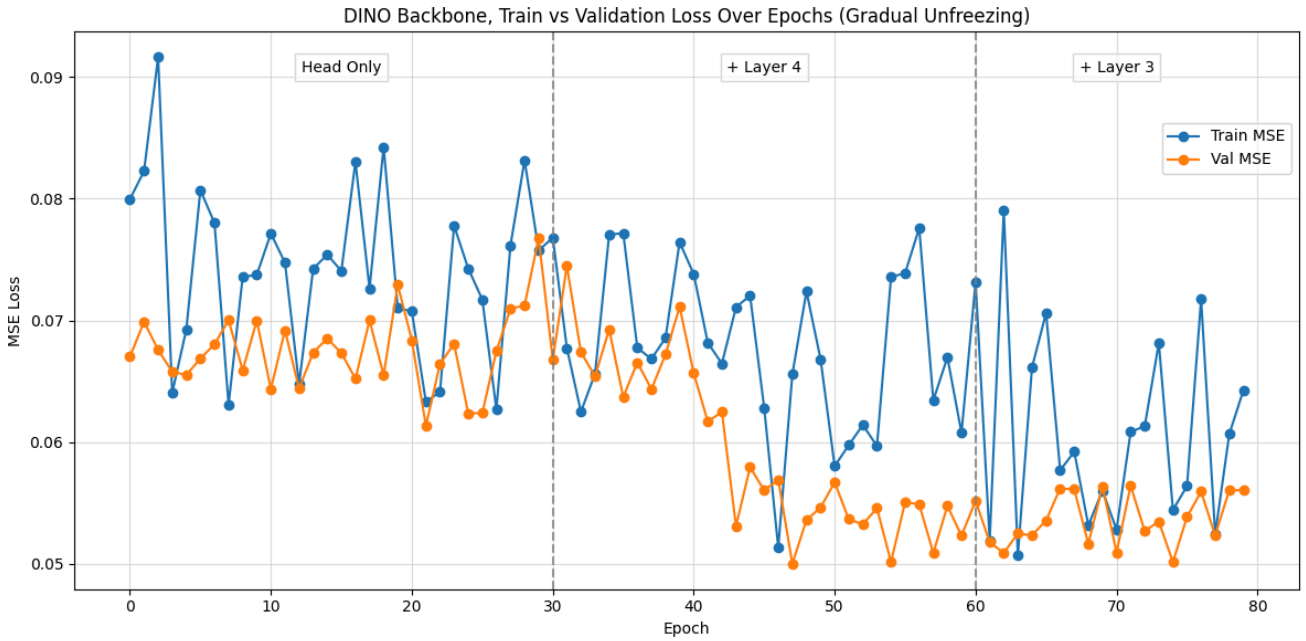

```
# After phase 3 training converges, unfreeze JUST layer 4
#FREEZE layer 4 + head when entering training
for param in resnet50.parameters(): #freezes all others
  param.requires_grad = False
for param in resnet50.layer3.parameters():
  param.requires_grad = True
for param in resnet50.layer4.parameters():
  param.requires_grad = True
for param in resnet50.fc.parameters():
  param.requires_grad = True

# Use a much lower LR than phase 1
optimizer = torch.optim.Adam([
    {'params': resnet50.layer3.parameters(), 'lr': 1e-5},
    {'params': resnet50.layer4.parameters(), 'lr': 1e-5},
    {'params': resnet50.fc.parameters(),     'lr': 1e-4},  # head can tolerate slightly higher LR
])
```

```
# TODO: Training w/IMAGENET-50 DINO backbone
from datetime import datetime
from torch.utils.tensorboard import SummaryWriter
import glob

# --- Resume from latest checkpoint if available ---
checkpoint_files = glob.glob('model_*')
if checkpoint_files:
    latest_checkpoint = max(checkpoint_files, key=lambda x: int(x.split('_')[-1]))
    print(f"Resuming from checkpoint: {latest_checkpoint}")
    resnet50.load_state_dict(torch.load(latest_checkpoint, map_location=device))
```

```
        # parse epoch number from filename (format: model_{timestamp}_{epoch})
        epoch_number = int(latest_checkpoint.split('_')[-1]) + 1
        timestamp = '_'.join(latest_checkpoint.split('_')[1:3])  # preserve original timestamp
        print(f"Resuming from epoch {epoch_number}")
    else:
        print("No checkpoint found, starting from scratch.")
        epoch_number = 0
        timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')

    writer = SummaryWriter('runs/fashion_trainer_{}'.format(timestamp))

    EPOCHS = 20

    best_vloss = 1_000_000.

    # val_losses = []
    # train_losses = []

    for epoch in range(EPOCHS):
        print('EPOCH {}:'.format(epoch_number + 1))

        # Make sure gradient tracking is on, and do a pass over the data
        resnet50.train(True)
        #REFREEZE layer 4 + head when entering training
        for param in resnet50.parameters(): #freezes all others
          param.requires_grad = False
        for param in resnet50.layer3.parameters():
          param.requires_grad = True
        for param in resnet50.layer4.parameters():
          param.requires_grad = True
        for param in resnet50.fc.parameters():
          param.requires_grad = True

        avg_loss = train_one_epoch(epoch_number, writer, optimizer)


        running_vloss = 0.0
        # Set the model to evaluation mode, disabling dropout and using population
        # statistics for batch normalization.
        resnet50.eval()

        # Disable gradient computation and reduce memory consumption.
        with torch.no_grad():
            # Wrap the validation loader
            val_pbar = tqdm(enumerate(test_loader), total=len(test_loader), desc="Validation")

            for i, vdata in val_pbar:
                vinputs, vlabels = vdata['image'], vdata['keypoints']
                vinputs = vinputs.repeat(1, 3, 1, 1)  # (B, 1, 224, 224) -> (B, 3, 224, 224)
                vinputs = vinputs.to(device)
                vlabels = vlabels.to(device)

                voutputs = resnet50(vinputs)
                vloss = loss_fn(voutputs, vlabels)
                running_vloss += vloss.item() # Use .item() to keep memory clean

        avg_vloss = running_vloss / (i + 1)
        print('LOSS train {} valid {}'.format(avg_loss, avg_vloss))
        train_losses.append(avg_loss)
        val_losses.append(avg_vloss)

        # Log the running loss averaged per batch
        # for both training and validation
        writer.add_scalars('Training vs. Validation Loss',
                        { 'Training' : avg_loss, 'Validation' : avg_vloss },
                        epoch_number + 1)
        writer.flush()

        # Track best performance, and save the model's state
        if avg_vloss < best_vloss:
            best_vloss = avg_vloss
            model_path = 'model_{}_{}'.format(timestamp, epoch_number)
            torch.save(resnet50.state_dict(), model_path)

        epoch_number += 1
```

```
Resuming from checkpoint: model_20260227_035648_38
Resuming from epoch 39
EPOCH 40:
Epoch 40 Training: 100%  217/217 [00:29<00:00,  8.84it/s, loss=0.0588]
Validation: 100%  49/49 [00:03<00:00, 16.53it/s]
LOSS train 0.05278926985895353 valid 0.050903884891233264
EPOCH 41:
Epoch 41 Training: 100%  217/217 [00:26<00:00,  8.82it/s, loss=0.0393]
Validation: 100%  49/49 [00:03<00:00, 18.30it/s]
LOSS train 0.0608264059761716 valid 0.05640728048810744
EPOCH 42:
Epoch 42 Training: 100%  217/217 [00:25<00:00,  8.93it/s, loss=0.1504]
Validation: 100%  49/49 [00:03<00:00, 18.71it/s]
LOSS train 0.061296595331936675 valid 0.052675064320710105
EPOCH 43:
Epoch 43 Training: 100%  217/217 [00:25<00:00,  8.96it/s, loss=0.0371]
Validation: 100%  49/49 [00:03<00:00, 15.70it/s]
LOSS train 0.068141029957382 valid 0.053455634423849256
EPOCH 44:
Epoch 44 Training: 100%  217/217 [00:25<00:00,  8.84it/s, loss=0.2846]
Validation: 100%  49/49 [00:03<00:00, 18.57it/s]
LOSS train 0.05440897019767209 valid 0.05012671318324115
EPOCH 45:
Epoch 45 Training: 100%  217/217 [00:25<00:00,  8.90it/s, loss=0.0331]
Validation: 100%  49/49 [00:03<00:00, 16.04it/s]
LOSS train 0.05637370907921966 valid 0.05383443319404591
EPOCH 46:
Epoch 46 Training: 100%  217/217 [00:25<00:00,  8.68it/s, loss=0.1484]
Validation: 100%  49/49 [00:03<00:00, 18.08it/s]
LOSS train 0.07174140257173159 valid 0.055919969913915096
EPOCH 47:
Epoch 47 Training: 100%  217/217 [00:25<00:00,  8.86it/s, loss=0.0262]
Validation: 100%  49/49 [00:03<00:00, 18.47it/s]
LOSS train 0.05240334477882045 valid 0.05231050279127037
EPOCH 48:
Epoch 48 Training: 100%  217/217 [00:25<00:00,  8.81it/s, loss=0.0786]
Validation: 100%  49/49 [00:03<00:00, 18.32it/s]
LOSS train 0.06064558776157938 valid 0.05604207625102987
EPOCH 49:
Epoch 49 Training: 100%  217/217 [00:26<00:00,  8.84it/s, loss=0.0454]
Validation: 100%  49/49 [00:03<00:00, 18.54it/s]
LOSS train 0.06424494899403523 valid 0.05602737203985916
```

Show code




Show code

```
Found checkpoints: ['model_20260227_035648_0', 'model_20260227_035648_3', 'model_20260227_035648_4', 'model_20260227_035648_10',
Selected: [('Earliest', 'model_20260227_035648_0'), ('Middle', 'model_20260227_035648_23'), ('Latest', 'model_20260227_035648_43
```


## Part 3: Heatmap-based Keypoint Detection

Show code

Show hidden output

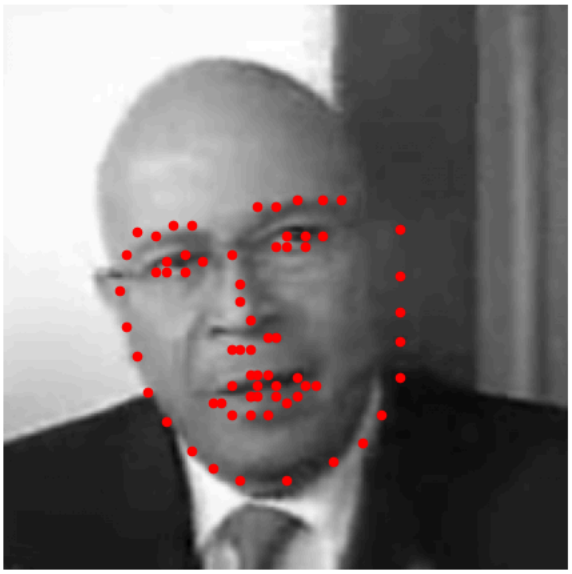
```
# importing the required libraries
import glob
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.image as mpimg
import cv2
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import transforms, utils
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
import numpy as np
from torch.utils.data import Dataset, DataLoader
from torch.utils.data.sampler import SubsetRandomSampler
from torch.optim import lr_scheduler
import torch.optim as optim
from tqdm import tqdm


# the transforms we defined in Notebook 1 are in the helper file `custom_transforms.py`
from custom_transforms import (
    Rescale,
    RandomCrop,
    Normalize,
    ToTensor,
)

# the dataset we created in Notebook 1
from facial_keypoints_dataset import FacialKeypointsDataset, FacialKeypointsHeatmapDataset

# defining the data_transform using transforms.Compose([all tx's, . , .])
# order matters! i.e. rescaling should come before a smaller crop
data_transform = transforms.Compose(
    [Rescale(250), RandomCrop(224), Normalize(), ToTensor()]
)

training_keypoints_csv_path = os.path.join("data", "training_frames_keypoints.csv")
training_data_dir = os.path.join("data", "training")
test_keypoints_csv_path = os.path.join("data", "test_frames_keypoints.csv")
test_data_dir = os.path.join("data", "test")


# create the transformed dataset
transformed_dataset = FacialKeypointsHeatmapDataset(
    csv_file=training_keypoints_csv_path,
    root_dir=training_data_dir,
    transform=data_transform,
)

# load training data in batches
batch_size = 16
train_loader = DataLoader(
    transformed_dataset, batch_size=batch_size, shuffle=True, num_workers=4
)

# creating the test dataset
test_dataset = FacialKeypointsHeatmapDataset(
    csv_file=test_keypoints_csv_path,
    root_dir=test_data_dir,
    transform=data_transform
)

# loading test data in batches
batch_size = 16
test_loader = DataLoader(
    test_dataset, batch_size=batch_size, shuffle=True, num_workers=4
)
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:424: UserWarning: This DataLoader will create 4 worker pr
  self.check_worker_number_rationality()
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:424: UserWarning: This DataLoader will create 4 worker pr
  self.check_worker_number_rationality()
```

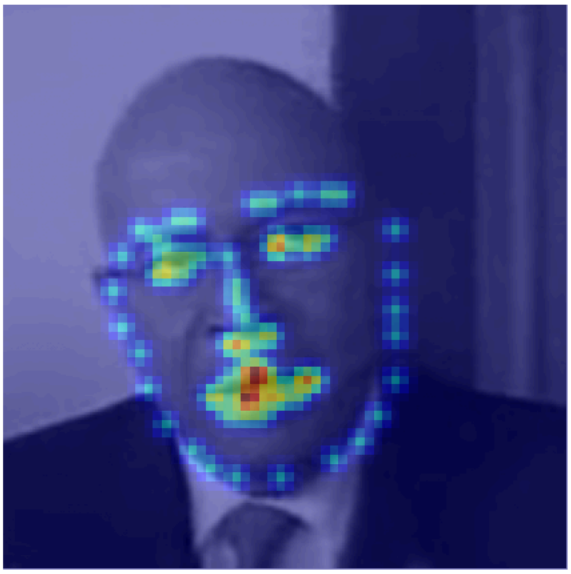
```
# 4.1
# TODO: Place a Gaussian centered at the keypoint location
# TODO: Generate heatmaps of lower resolution than the input image (e.g. 64x64)
# TODO: Ensuring proper normalization of the heatmaps

example_idx = 0
for i, data in enumerate(test_loader):
    sample = data
    image = sample['image'][example_idx]
    print("Image shape: ", image.shape)
    keypoints = sample['keypoints'][example_idx]
    heatmap = sample['heatmaps'][example_idx]
    print("Heatmaps shape: ", sample['heatmaps'].shape)

    img_size = image.shape[-1]  # 224

    # Collapse the 68 heatmap channels into a single 2D representation by
    # summing together heatmaps. Note: using max instead of sum can help
    # overlapping Gaussian blobs look crispier
    heatmap_2d = torch.sum(heatmap, dim=0).numpy()

    heatmap_2d = heatmap_2d - heatmap_2d.min() # heatmap normalization [0, 1]
    heatmap_2d = heatmap_2d / heatmap_2d.max()
    print("Shape of heatmap_2d: ", heatmap_2d.shape)

    # Set up a 1x2 grid of plots
    fig, axes = plt.subplots(1, 2, figsize=(12, 6))

    # Ground Truth Keypoints: Image + Keypoint Scatter
    axes[0].imshow(image.numpy().transpose(1, 2, 0), cmap='gray')
    axes[0].scatter(
        keypoints[:, 0] * (img_size / 4.0) + (img_size / 2.0),
        keypoints[:, 1] * (img_size / 4.0) + (img_size / 2.0),
        c='r', s=20
    )
    axes[0].set_title("Ground Truth Keypoints")
    axes[0].axis('off')

    # Target Heatmaps: Image + Heatmap Overlay
    axes[1].imshow(image.numpy().transpose(1, 2, 0), cmap='gray')

    # Overlay the heatmap.
    # 'jet' or 'hot' are great colormaps for this.
    # We use extent=[0, img_size, img_size, 0] to force the heatmap to stretch over the
    # 224x224 image perfectly, just in case your model output a smaller heatmap (e.g. 56x56)
    axes[1].imshow(heatmap_2d, cmap='jet', alpha=0.5, extent=[0, img_size, img_size, 0])
    axes[1].set_title("Target Heatmaps")
    axes[1].axis('off')

    plt.show()
    break
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:432: UserWarning: This DataLoader will create 4 worker pr
  self.check_worker_number_rationality()
Image shape:  torch.Size([1, 224, 224])
Heatmaps shape:  torch.Size([16, 68, 64, 64])
Shape of heatmap_2d:  (64, 64)
```

Ground Truth Keypoints



Target Heatmaps



```
# TODO: Heatmap synthesis and training

# 4.2 U-Net Heatmap prediction
class doubleConv(nn.Module):
    def __init__(self, in_channels, out_channels):
        super(doubleConv, self).__init__()
        self.conv = nn.Sequential(
          # output_dim = ((input_dim - kernel_size + 2*padding) / stride) + 1
          # keep spatial dimensions the same (since start dims not perfect square)
          nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=1, padding=1),
          nn.BatchNorm2d(out_channels),
          nn.ReLU(inplace=True),
          # keep spatial dimensions the same
          nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1, padding=1),
          nn.BatchNorm2d(out_channels),
          nn.ReLU(inplace=True)
        )

    def forward(self, x):
        return self.conv(x)

class HeatMapPredUNet(nn.Module):

    def __init__(self):
        super(HeatMapPredUNet, self).__init__()
        # Inputs:
          # Image shape:  torch.Size([1, 224, 224])
        # Outputs:
          # HeatMap shape:  torch.Size([K, 64, 64]),
          # where K = 68 (the number of face feature points)

        # DOWN-block
        self.doubleConv1 = doubleConv(in_channels=1, out_channels=64)
        self.doubleConv2 = doubleConv(in_channels=64, out_channels=128)
        self.doubleConv3 = doubleConv(in_channels=128, out_channels=256)
        self.doubleConv4 = doubleConv(in_channels=256, out_channels=512)
        #self.doubleConv5 = nn.doubleConv(in_channels=512, out_channels=1024)

        # UP-block
        #self.doubleConv6 = nn.doubleConv(in_channels=1024, out_channels=512)
        self.doubleConv5 = doubleConv(in_channels=256, out_channels=512)
        self.doubleConv6 = doubleConv(in_channels=512, out_channels=256)
        self.doubleConv7 = doubleConv(in_channels=256, out_channels=128)
        self.doubleConv8 = doubleConv(in_channels=128, out_channels=64)
```

```
        # pooling function
        self.pool = nn.MaxPool2d(2, 2) #(size, stride)

        # up-conv function
        #self.upconv1 = nn.ConvTranspose2d(1024, 512, kernel_size=2, stride=2, padding=0)
        self.upconv1 = nn.ConvTranspose2d(512, 256, kernel_size=2, stride=2, padding=0)
        self.upconv2 = nn.ConvTranspose2d(256, 128, kernel_size=2, stride=2, padding=0)
        self.upconv3 = nn.ConvTranspose2d(128, 64, kernel_size=2, stride=2, padding=0)

        # conv 1x1 (which I think would just be redundently copying it over)
        self.conv_out = nn.Conv2d(64, 68, kernel_size=1, stride=1, padding=0)

        # sigmoid function
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        # x: (1, 224, 224)

        # Down-block
        skip1 = self.doubleConv1(x) # (64, 222, 222)
        x = self.pool(skip1) # (64, 112, 112)

        skip2 = self.doubleConv2(x) # (128, 112, 112)
        x = self.pool(skip2) # (128, 56, 56)

        skip3 = self.doubleConv3(x) # (256, 56, 56)
        x = self.pool(skip3) # (256, 28, 28)

        # skip4 = self.doubleConv4(x)
        # x = self.pool(skip4)

        # Bottom of U
        x = self.doubleConv5(x) # (512, 28, 28)

        # UP-block
        x = self.upconv1(x) # (256, 56, 56)
        x = torch.cat((skip3, x), dim=1) # (512, 56, 56)
        x = self.doubleConv6(x) # (256, 56, 56)

        x = self.upconv2(x) # (128, 112, 112)
        x = torch.cat((skip2, x), dim=1) # (256, 112, 112)
        x = self.doubleConv7(x) # (128, 112, 112)

        x = self.upconv3(x) # (64, 224, 224)
        x = torch.cat((skip1, x), dim=1) # (128, 224, 224)
        x = self.doubleConv8(x) # (64, 224, 224)

        # x = self.upconv4(x)
        # x = torch.cat((skip1, x), dim=1)
        # x = self.doubleConv9(x)

        # Force the output down to 64x64: (64, 64, 64)
        x = F.interpolate(x, size=(64, 64), mode='bilinear', align_corners=False)

        x = self.conv_out(x) # (68, 64, 64)

        x = self.sigmoid(x) # bound (normalize) values between 0 and 1
        return x

model = HeatMapPredUNet()
print(model)
```

Show hidden output

```
loss_fn = nn.MSELoss()
```

```
# This will use the GPU if available, otherwise it falls back to CPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

model = model.to(device)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-2)
```

```
Using device: cuda
```

```
from tqdm.auto import tqdm
def train_one_epoch(epoch_index, tb_writer, optimizer):
    running_loss = 0.
    last_loss = 0.

    # Wrap the loader with tqdm
    # Here, we use enumerate(training_loader) instead of
    # iter(training_loader) so that we can track the batch
    pbar = tqdm(enumerate(train_loader), total=len(train_loader), desc=f"Epoch {epoch_index + 1} Training")

    # index and do some intra-epoch reporting
    for i, data in pbar:
        # Every data instance is an input + label pair
        sample = data
        inputs, labels, heatmaps = sample['image'], sample['keypoints'], sample['heatmaps']
        inputs = inputs.to(device)
        labels = labels.to(device)
        heatmaps = heatmaps.to(device)

        # Zero your gradients for every batch!
        optimizer.zero_grad()

        # Make predictions for this batch
        outputs = model(inputs)

        # Compute the loss and its gradients
        loss = loss_fn(outputs, heatmaps)
        loss.backward()

        # Adjust learning weights
        optimizer.step()

        # Gather data and report
        running_loss += loss.item()

        # Update the progress bar with the current loss
        pbar.set_postfix({'loss': f"{loss.item():.4f}"})

        if i % 10 == 9:
            last_loss = running_loss / 10 # loss per batch
            #print('  batch {} loss: {}'.format(i + 1, last_loss))
            tb_x = epoch_index * len(train_loader) + i + 1
            tb_writer.add_scalar('Loss/train', last_loss, tb_x)
            running_loss = 0.

    return last_loss
```

```
# TODO: Training a simple CNN
from datetime import datetime
from torch.utils.tensorboard import SummaryWriter
import glob

# --- Resume from latest checkpoint if available ---
checkpoint_files = glob.glob('model_*')
if checkpoint_files:
    latest_checkpoint = max(checkpoint_files, key=lambda x: int(x.split('_')[-1]))
    print(f"Resuming from checkpoint: {latest_checkpoint}")
    model.load_state_dict(torch.load(latest_checkpoint, map_location=device))
    # parse epoch number from filename (format: model_{timestamp}_{epoch})
    epoch_number = int(latest_checkpoint.split('_')[-1]) + 1
    timestamp = '_'.join(latest_checkpoint.split('_')[1:3])  # preserve original timestamp
    print(f"Resuming from epoch {epoch_number}")
else:
    print("No checkpoint found, starting from scratch.")
    epoch_number = 0
    timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')

writer = SummaryWriter('runs/fashion_trainer_{}'.format(timestamp))

EPOCHS = 40

best_vloss = 1_000_000.

val_losses = []
train_losses = []

for epoch in range(EPOCHS):
```

# References


[1] Mathilde Caron, Hugo Touvron, Ishan Misra, Hervé Jégou, Julien Mairal, Piotr Bojanowski, and Armand Joulin. Emerging properties in self-supervised vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9650–9660, 2021.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

[3] PyTorch Contributors. Training a classifier — pytorch tutorials. https://docs.pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html, 2026. Accessed: 2026-02-25.

[4] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.